arXiv:2609.05533v1 [cs.CV] 2 Sep 2026

# SIMPLEMEMVLA: A SIMPLE BUT EFFECTIVE NATIVE-VIDEO MEMORY FOR VISION-LANGUAGE-ACTION MODELS

**Cheng Yin**[1,8] **Wang Xu**[3] **Junpeng Yang**[6] **Sikyuen Tam**[3,6] **Hanyu Liu**[5] **Yuan Yao**[4]
**Xiangrui Zeng**[1*] **Junbo Cui**[6*] **Yequan Wang** [7] **Zhouping Yin**[1] **Yankai Lin** [2,7*]
[1]Research Center for Advanced Electronics Manufacturing, MSE†, HUST‡
[2]Gaoling School of Artificial Intelligence, Renmin University of China
[3]Department of Computer Science and Technology, Tsinghua University
[4]College of AI, Tsinghua University [5]Peking University [6]Modelbest
[7]Beijing Academy of Artificial Intelligence [8]Zhongguancun Academy, Beijing, China
yinchenghust@hust.edu.cn yankailin@ruc.edu.cn zeng@hust.edu.cn

## ABSTRACT

Long-horizon manipulation is partially observable: the information needed to choose the next action may appear only in observations from minutes earlier. Existing memory mechanisms: retrieval banks, learned compressors, recurrent states must decide what to keep from the past before knowing what a future decision will require. This was motivated by the assumption that minute-scale history is too large to process directly, which modern VLM backbones no longer make true. In this work, we introduce SimpleMemVLA, a VLA without a dedicated memory module. It keeps the sampled history intact and passes it to the backbone in the timestamped video format the backbone was pretrained to process; the hidden states of a generated sub-task then form the only channel from history to a standard flow-matching action head. Since consecutive decisions share most of their history, prefilling the shared prefix during action execution keeps latency close to a single-frame VLA. SimpleMemVLA sets a new state of the art on four memory benchmarks without cost on general-purpose control. Holding the backbone and training setup fixed, it outperforms retrieval, compression and recurrent-state mechanisms by a wide margin, and causal interventions confirm that the policy genuinely reads its history.[1]

## 1 INTRODUCTION

As vision-language-action (VLA) models move from short tabletop skills to long-horizon tasks, partial observability becomes unavoidable (Kaelbling et al., 1998): information needed to choose the next action may appear only in observations from minutes earlier (Ma et al., 2024; Sapkota et al., 2025; Shi et al., 2026a; Koo et al., 2025). A robot may need to remember which object was revealed, where an occluded target was placed, or how many times an action has already been completed. Most general-purpose VLAs, however, condition their actions on a single image or a sub-second observation window (Brohan et al., 2023; Team et al., 2024; Kim et al., 2024; Black et al., 2025; Intelligence et al., 2025; Liu et al., 2024b; Nvidia et al., 2025). Such policies cannot solve these tasks no matter how well it is trained, since two states with identical current observations may require different actions.

Existing work provides memory through dedicated mechanisms (Figure 1): retrieval banks that select observations from an external store (Memmel et al., 2025; Li et al., 2025; Sridhar et al., 2026; Hu et al., 2026; Lin et al., 2025a; Lei et al., 2025), learned compressors that summarize history

*Corresponding author: Xiangrui Zeng, Junbo Cui and Yankai Lin
†MSE: School of Mechanical Science and Engineering
‡HUST: Huazhong University of Science and Technology
[1]Code available at https://github.com/wadeKeith/SimpleMemVLA

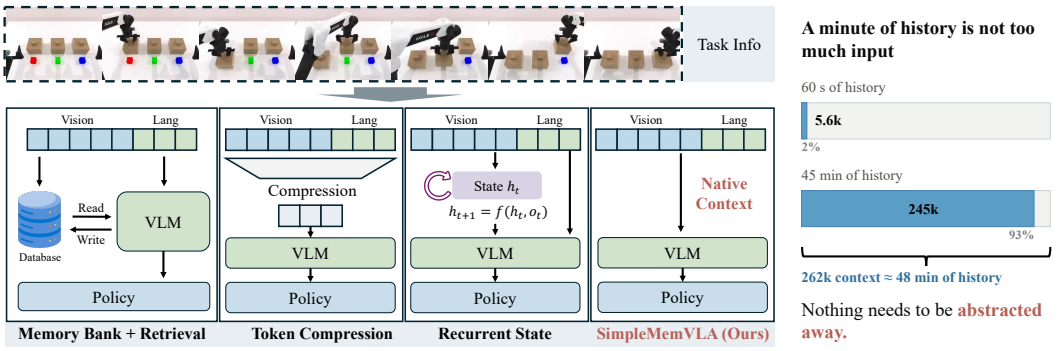



Figure 1: **VLA memory mechanisms versus SimpleMemVLA.** *Left:* Three prior design families insert dedicated memory machinery between the observation stream and policy, while SimpleMemVLA uses the timestamped stream directly as native context. *Right:* A 60 s history uses only 5.6k tokens of the backbone's 262k-token context window, which can hold roughly 45 minutes.

within a fixed token budget (Shi et al., 2026a; Jang et al., 2025; Wang et al., 2026b; Koo et al., 2025), and recurrent states that continually update a compact representation of the history (Li et al., 2026; Cherepanov et al., 2026; Liu et al., 2024a; Qu et al., 2026). Although these mechanisms differ in implementation, each must determine what remains available from the history before the needs of a future decision are known. A relevant frame may be omitted during retrieval, visual details lost during compression, or earlier evidence overwritten by a recurrent update. The discarded information may become relevant only later. We refer to this as *write-time commitment*.

The dedicated memory mechanisms described above were motivated by the assumption that minute-scale history was too large to process directly, but that assumption no longer holds. Modern VLM backbones are pretrained to process temporally ordered video through native multimodal interfaces (Wang et al., 2024; Bai et al., 2025b; Yang et al., 2025; Bai et al., 2025a) and can read timestamped streams directly. At the sampling rates used for manipulation, a 60 s history occupies roughly 5.6k tokens of a 262k-token context window. For histories at this scale, context capacity alone no longer requires the past to be compressed into a separate memory representation.

We propose **SimpleMemVLA**, a VLA without a dedicated memory module that uses the pretrained backbone's native video context directly as memory. The challenge is not whether the history can be provided as input, but whether the policy can use it without slowing the control loop: raw observations must reach the backbone in a form it can process, the evidence it finds there must drive the action head, and none of this may push the decision past its real-time budget. SimpleMemVLA keeps the sampled history intact until the current decision and presents it in the same timestamped video format that the backbone was pretrained to process (Bai et al., 2025a). At that point, the backbone identifies the evidence relevant to the decision directly from the provided history, rather than relying on an earlier choice about what information to retain. The backbone then generates the current textual sub-task. The contextual hidden states and token embeddings of this span provide the only channel through which historical information reaches a standard flow-matching action head (Lipman et al., 2022; Black et al., 2025; Intelligence et al., 2025; Nvidia et al., 2025). Consecutive decisions share most of their visual history, so we prefill the shared history prefix while the robot executes the current action chunk and reuse it at the next decision. This reduces decision latency to 0.68 s, close to that of a single-frame VLA, while producing the same outputs as full recomputation.

Our experiments show the following: First, no dedicated memory mechanism is needed. SimpleMemVLA sets a new state of the art on all four memory benchmarks while maintaining performance comparable to the strongest reactive VLAs on general-purpose control tasks, indicating that carrying a long visual history does not reduce performance when memory is not required. Second, these gains cannot be explained by the backbone alone. Re-implementing retrieval, compression, and recurrent-state methods with the same backbone and training setup, we find that native context reaches 88.3% on RoboMME, while the three alternatives reach 31.5%, 22.6%, and 20.6%, respectively. Even with ground-truth perception, performance reaches only 84.1%, so the limitation lies in when these mechanisms commit information, rather than how accurately they do so. Third, further

analyses show that the policy uses information from its visual history: masking task-relevant evidence changes its outputs, whereas masking an irrelevant segment does not. The policy also adapts its behavior to edited or previously unseen visual histories without parameter updates, indicating a form of visual in-context learning (Brown et al., 2020; Alayrac et al., 2022).

## 2 Related Work

### 2.1 Vision-Language-Action Models

Most research on generalist VLAs has focused on improving how policies map the observations available at the current decision to actions. This work spans two broad directions. Research on *action generation* has progressed from co-fine-tuned VLMs with discretized actions (Brohan et al., 2023; Kim et al., 2024; Hung et al., 2025) and control-specific tokenizers (Pertsch et al., 2025; Kim et al., 2025) to continuous diffusion and flow-matching experts (Chi et al., 2023; Lipman et al., 2022; Team et al., 2024; Liu et al., 2024b; Black et al., 2025). Research on *policy architecture and capability* has explored hierarchical or dual-system designs (Nvidia et al., 2025; Intelligence et al., 2025), spatial and trace representations (Qu et al., 2025; Zheng et al., 2025), video-pretrained world models (Cheang et al., 2024; Cen et al., 2025; Guo et al., 2025), reasoning and interactive post-training (Yin et al., 2026; Tan et al., 2025), and cross-embodiment transfer (Zheng et al., 2026; Bu et al., 2025). These advances have improved both VLA capabilities and action generation, but how a policy should process minute-scale execution history remains an open question (Ma et al., 2024; Sapkota et al., 2025). SimpleMemVLA addresses this question by testing whether a pretrained backbone can process timestamped visual history directly through its native video channel without a dedicated memory mechanism.

### 2.2 Memory Mechanisms for VLAs

As VLAs are deployed in longer, partially observable tasks (Kaelbling et al., 1998), the information needed for a decision may be available only in earlier observations, making it increasingly important to preserve and reuse visual history (Ma et al., 2024; Sapkota et al., 2025; Shi et al., 2026a; Koo et al., 2025). Existing memory designs fall into four families, distinguished by what they preserve at write time before the requirements of a future decision are known: symbolic storage (Sun et al., 2026a; Huang et al., 2026; Lei et al., 2025), retrieval (Memmel et al., 2025; Sridhar et al., 2026; Yang et al., 2026a), learned compression (Shi et al., 2026a; Jang et al., 2025; Wang et al., 2026b), and recurrent state (Cherepanov et al., 2026; Li et al., 2026; Qu et al., 2026).

*Symbolic* pipelines provide the most explicit representation, parsing observations into structured stores outside the policy, such as scene graphs, concept banks, and execution states (Dai et al., 2026; Sun et al., 2026a; Huang et al., 2026). Because observations are written into a predefined schema, information outside its vocabulary is discarded during extraction, even with perfect perception. *Retrieval* methods preserve experience in an external store but expose only selected content to the policy, such as sub-trajectories, retrieved experiences, or event evidence (Memmel et al., 2025; Sridhar et al., 2026; Yang et al., 2026a). Fixed sampling schedules can be viewed as a degenerate case (Lin et al., 2026). Because the retrieval index is constructed before the current query is available, frames that are not retrieved, along with their order and timestamps, remain unavailable to the policy for that decision. *Compression* methods instead map history into bounded learned representations, such as consolidated memory banks, amortized context tokens, or compressed visual features (Shi et al., 2026a; Jang et al., 2025; Wang et al., 2026b). Because the representation budget is fixed at observation time, the method must determine which perceptual details to retain before their relevance to a future decision is known. *Recurrent* methods maintain a bounded summary of the past as a continually updated state, using recurrent tokens, latent memories, or gated updates (Cherepanov et al., 2026; Qu et al., 2026; Gao et al., 2026). At each update, the method must decide what to overwrite before future needs are known; once overwritten, that evidence can no longer be recovered from the state. Despite these differences, all four families give the policy access to past observations through an intermediate memory interface: a symbolic store, a retrieval index, a compressed representation, or a recurrent state. MEM (Torne et al., 2026), the design closest to ours, compresses both recent frames and older events before they reach the backbone, preventing it from attending directly to past frames. It targets settings where processing the full history exceeds the real-time budget; we instead study minute-scale histories that fit within the context window. SimpleMemVLA uses no such inter-

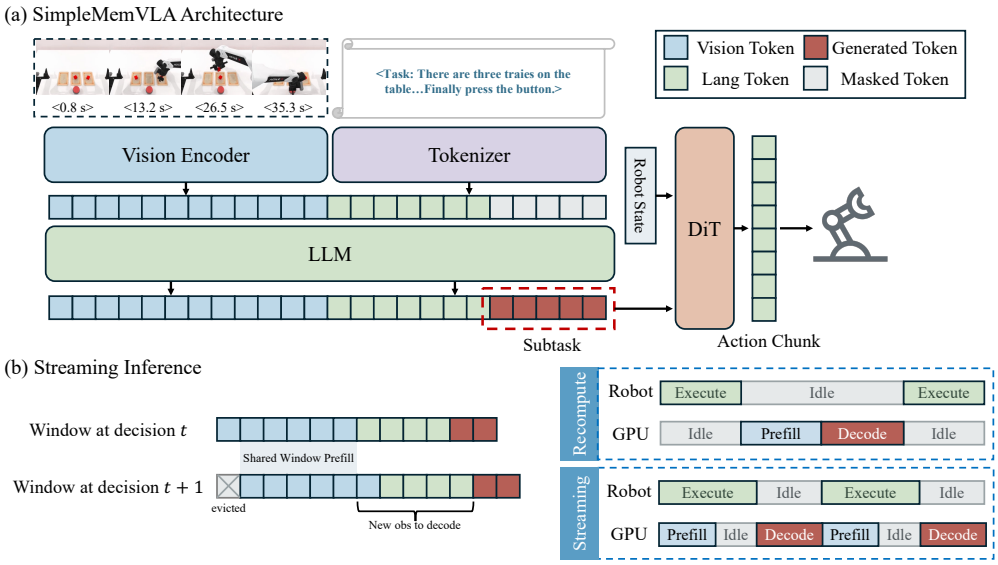



Figure 2: **SimpleMemVLA architecture and streaming inference.** *(a)* The architecture uses only standard VLA components, with self-attention over plaintext-timestamped history serving as memory. *(b)* Consecutive decisions differ by only one temporal patch, enabling shared-prefix prefill during action execution and reducing latency from 1.02 s to 0.68 s with identical outputs (Section 4.5).

face and instead presents minute-scale timestamped video history directly to the backbone, allowing native attention to select the past evidence relevant to each decision. A similar result has been reported in streaming video understanding, where an off-the-shelf VLM given a sliding window of recent frames matches or outperforms dedicated streaming-memory methods (Shen et al., 2026).

# 3 SimpleMemVLA

## 3.1 Problem Setup and Overview

Memory-dependent manipulation is a partially observable control problem: the current observation alone may not determine the correct action. At step $t$, given a language instruction $\ell$, the robot receives an observation $o_t$ consisting of camera images and a proprioceptive state $q_t$, and outputs an action $a_t$. On the benchmarks of Section 4, two states with identical $o_t$ can demand different actions depending on events minutes in the past, such as which mat a block was lifted from or how many times a button has already been pressed, so any reactive policy $\pi(a_t \mid o_t, \ell)$ is ill-posed no matter how well it is trained. The policy must condition on the history $h_t = (o_{\le t}, \ell)$. The central design question is how the policy should access this history: through dedicated memory machinery or directly as native video context. At each decision, the policy states the current sub-task $g_t$, a short textual description of what should be done now. During training, this output is supervised by a target $g_t^*$ whose construction is described in Appendix A.

Figure 2 shows the architecture: a window of past head-camera frames enters the Qwen3.5-4B backbone through its native video channel, current wrist views enter as images and the backbone states the current sub-task in text, whose hidden states condition a flow-matching head that produces an action chunk. The next three sections give, in turn, the format in which history is retained for the backbone to read (Section 3.2), the channel that turns what is read into actions (Section 3.3) and the inference scheme that keeps this retention affordable at deployment (Section 3.4).

## 3.2 Retaining History for Read-Time Selection

This section specifies the form in which history is retained so that self-attention can select from it at read time. The backbone's pretraining already fixes the right form: video with frame order and

plaintext timestamps carries temporal grounding natively, whereas any non-native packing, such as concatenating frames as separate images, asks the model to relearn temporal structure from scratch. Let $f_c$ be the native frame rate of the observation stream and $o_t^{\mathrm{h}}$ the head-camera frame at step $t$. SimpleMemVLA keeps no state across steps but rebuilds, at every prediction step, a window covering the last $T_w$ seconds subsampled at a rate $f_v \ll f_c$ into at most $K = T_w f_v$ frames,

$$V_t \;=\; \big(o_{t-(K-1)s}^{\mathrm{h}},\, \ldots,\, o_{t-s}^{\mathrm{h}},\, o_t^{\mathrm{h}}\big), \qquad s = f_c/f_v, \tag{1}$$

where $s$ is the subsampling stride and an episode younger than $T_w$ simply yields a shorter clip. Per suite, $T_w$ is set to cover the horizon over which its tasks leave evidence and $f_v$ is the lowest rate that does not skip decisive events, trading token budget against coverage (Table 7). $V_t$ enters the backbone through its *video* channel, whose processor groups adjacent frames into temporal patches and prefixes each patch with the backbone's native plaintext timestamp, exactly as in video pretraining (Bai et al., 2025a). Standard deployment uses window-relative timestamps, labeling each patch by its offset within the active window, whereas the exploratory SWA variant in Appendix E uses episode-absolute timestamps to keep cached patches immutable during future continual operation over unbounded input streams. These timestamps provide the system's only temporal grounding by encoding each patch's position within the window in a format the backbone already understands, allowing it to locate observed events in time relative to the current decision.

The current wrist frames $\{o_t^{\mathrm{w},i}\}_{i=1}^{W}$, where $W$ is the number of wrist cameras, enter through the *image* channel without timestamps, under a single modality rule: multi-frame cameras become video and single-frame cameras become images. The rule lets the input format itself separate the past from the present, so the prompt, assembled by the prompt builder $\Phi$,

$$x_t \;=\; \Phi\big(V_t,\, \{o_t^{\mathrm{w},i}\}_{i=1}^{W},\, \ell\big) \tag{2}$$

The resulting prompt needs only a plain-text description of the embodiment, camera layout and timestamp convention. Low-rate sampling at $f_v$ keeps minute-scale history within a modest token budget, while an episode-scale window $T_w$ keeps early sampled evidence available even at the end of an episode. All sampled evidence within the window is therefore accessible to the backbone at read time, leaving the next question of how the resulting task state reaches the action expert.

### 3.3 A Narrow Text Channel from History to Action

This section describes how information selected from the visual history reaches the action expert. The expert accepts only a short token sequence rather than the thousands of visual tokens in the history window, so the backbone must distill the relevant information into a compact conditioning signal. SimpleMemVLA uses the generated sub-task span as this interface, yielding a signal that is compact, directly inspectable and editable. The backbone $f_\theta$, with parameters $\theta$, generates this span, while the DiT-style flow-matching expert $v_\phi$, with parameters $\phi$, conditions on its contextual representation and the current proprioceptive state. We define the generated sub-task, conditioning set and conditional flow-matching objective (Lipman et al., 2022) as:

$$\begin{aligned} g_t \;&=\; (g_{t,1}, \ldots, g_{t,m}) \;\sim\; f_\theta(\,\cdot \mid x_t), \\ C_t \;&=\; \big[\, e(g_{t,1}) \oplus h(g_{t,1}),\; \ldots,\; e(g_{t,m}) \oplus h(g_{t,m}),\; \psi(\bar{q}_t) \,\big], \\ \mathcal{L}_{\mathrm{act}} \;&=\; \mathbb{E}_{\tau,\,\varepsilon}\, \big\| v_\phi\big(A^\tau, \tau \mid C_t\big) - \big(\varepsilon - \bar{A}_t\big) \big\|_2^2 . \end{aligned} \tag{3}$$

Here, $g_t$ is a one-sentence description of the robot's current sub-task, generated as an ordinary assistant response under an unmodified chat template rather than as chain-of-thought. Decoding is capped at 64 tokens, keeping $m$ small. The function $h(\cdot)$ returns the backbone hidden states over this response, $e(\cdot)$ its corresponding token embeddings, $\oplus$ denotes their fusion and $\psi(\bar{q}_t)$ is a single-token encoding of the normalized current proprioceptive state. In the objective, $\bar{A}_t \in \mathbb{R}^{H \times d_a}$ denotes the normalized action chunk for the next $H$ steps, with each of its $d_a$ action dimensions z-scored using dataset statistics. We sample noise $\varepsilon \sim \mathcal{N}(0, I)$ and a flow time $\tau \in [0, 1]$ from a distribution biased toward the noise endpoint, then construct the linear path $A^\tau = (1 - \tau)\, \bar{A}_t + \tau\, \varepsilon$. Because the expert receives no prompt tokens directly, all history-dependent information needed for control must reach it through the generated answer span. Native attention over $V_t$ selects the relevant evidence and encodes it in the span's contextual representation, distilling minute-scale history into a compact interface that remains explicit and inspectable.

Training uses two forms of supervision. The action target is the demonstrated action chunk. The sub-task target $g_t^*$ is generated offline by a cloud LLM, which is given each demonstration and describes the sub-task underway at every supervision anchor (Appendix A). The same targets supervise all three mechanism variants in Section 4.3, ensuring that their within-stack comparison isolates the memory interface rather than annotation quality. The joint objective is

$$\mathcal{L} = \lambda_{\text{sub}}\mathcal{L}_{\text{sub}} + \lambda_{\text{act}}\mathcal{L}_{\text{act}}, \tag{4}$$

where $\mathcal{L}_{\text{sub}}$ is the token-level cross-entropy over the answer span. At deployment, the expert initializes an action chunk from Gaussian noise and integrates the learned velocity field toward a clean chunk using a small number of Euler steps. The resulting chunk is denormalized and its first $n_e$ actions are executed. Observations continue to be buffered at the native control rate during execution, allowing the window in Equation 1 to be reconstructed exactly for the next prediction.

### 3.4 Exact Streaming Inference

The final requirement is deployment efficiency: retaining minute-scale history should not place the cost of reprocessing the entire window on the critical path of every decision. Consecutive decisions share nearly the entire video-history prefix, so SimpleMemVLA prefills this shared prefix while the robot executes the current action chunk and stores the resulting key–value cache. At the next decision, the policy processes only the newly arrived temporal patch and the text instruction before decoding the next sub-task and action chunk. Overlapping history processing with action execution reduces decision-time latency without changing the policy output (Section 4.5).

The current scheme makes bounded, minute-scale native context practical for deployment. As a step toward continual inference over native video streams of unbounded duration, Appendix E explores a variant trained with sliding-window attention (SWA), which keeps its active context and cache bounded as the history grows.

Neither the native-context memory nor its streaming implementation depends on a particular robot. Within the architecture, embodiment-specific choices enter only through the configuration tuple $(\mathcal{C}\text{hist}, \mathcal{C}\text{cur}, T_w, f_v, H, d_a)$, which specifies the history and current camera sets, window length, sampling rate, action horizon and action dimensionality. Moving between bimanual and single-arm platforms therefore changes this configuration rather than the memory mechanism. Appendix A lists the concrete configuration used for each benchmark.

## 4 Experiments

We organize our experiments around five questions. First, **how effective is SimpleMemVLA?** We evaluate it on four memory-centric and two general-purpose benchmarks against published baselines (Section 4.2). Second, **does the gain come from the memory interface itself?** Holding everything else fixed, we rebuild one method from each mechanism family on our stack and compare them with native video context (Section 4.3). Third, **does the policy actually use its visual history as memory?** Holding the current observation and policy fixed, we remove or counterfactually replace the evidence in earlier frames and measure whether the output changes (Section 4.4). Fourth, **can long-context memory be deployed efficiently?** We evaluate streaming inference, which reuses the shared history across consecutive decisions instead of recomputing it (Section 4.5). Finally, **how does each element of the recipe contribute?** We ablate history length, frame order, plaintext timestamps and the hidden-state and token-embedding inputs to the action head (Section 4.6).

### 4.1 Experimental Setup

We evaluate SimpleMemVLA on six manipulation suites: four memory-centric benchmarks and two general-purpose controls. The four complement one another in embodiment (bimanual and single-arm), memory content (from counting and object permanence to occlusion and ordered sequences) and interaction horizon (from a few seconds to beyond one thousand control steps). **RMBench** (Chen et al., 2026) comprises ten memory-dependent bimanual tasks built on RoboTwin 2.0 (Chen et al., 2025) and organized into single-memory $M(1)$ and multi-memory $M(n)$ settings. Published baselines are available for nine tasks; because Place-Mat has so far

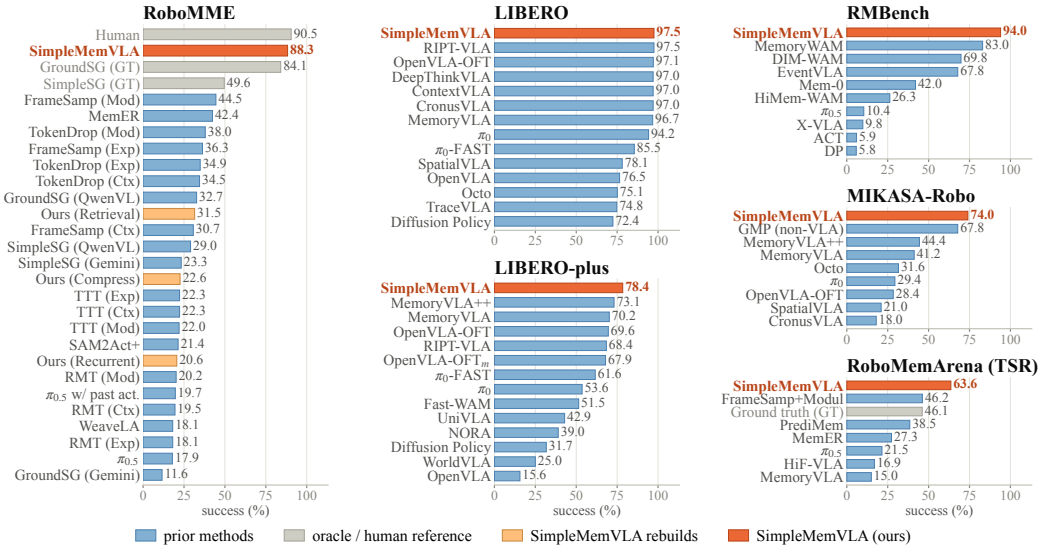



Figure 3: **SimpleMemVLA leads all memory suites and matches the best results on the general-purpose suites.** All methods from the per-suite tables are shown in score order. SimpleMemVLA uses one model per suite, while RMBench baselines are task-specific specialists. Gray oracle and human references are excluded from ranking.

been evaluated only by SimpleMemVLA, all cross-method averages are computed over the remaining nine. **RoboMME** (Dai et al., 2026) contains sixteen single-arm tasks spanning four memory dimensions: Counting, Permanence, Reference and Imitation. The benchmark also reports fourteen memory variants built on a shared $\pi_{0.5}$ backbone. **MIKASA-Robo** (Cherepanov et al., 2025) comprises five single-arm tasks designed to test memory under occlusion and partial observability. **RoboMemArena** (Lei et al., 2026) contains twenty-six single-arm tasks across four categories—Transferring, Occlusion, Counting and Sequence—with trajectories averaging more than one thousand control steps. Performance is measured by all-stage task success (TSR) and stage completion (CSR). To assess whether using long visual histories compromises general manipulation performance, we additionally evaluate on LIBERO (Liu et al., 2023) and test zero-shot transfer to LIBERO-Plus (Fei et al., 2025), which expands the four LIBERO suites into 10,030 perturbed tasks across seven perturbation dimensions. All reported results are obtained through closed-loop evaluation using a predefined seed set for each suite, disjoint from the seeds used to collect the training data. Suite-specific evaluation protocols are documented in Appendix A. For each suite, SimpleMemVLA is trained once and evaluated using a single model shared across all tasks in that suite. In contrast, each published RMBench baseline trains a separate task-specific specialist. All SimpleMemVLA evaluations use the streaming deployment described in Section 4.5.

### 4.2 Effectiveness: State-of-the-Art Performance Across Six Benchmarks

Figure 3 summarizes performance across all six suites. SimpleMemVLA achieves the highest reported performance on each of the four memory-centric benchmarks, ties the best reported average on the general-purpose benchmark LIBERO and achieving the strongest reported zero-shot transfer to its perturbation extension, LIBERO-Plus. Together, these results show that native context provides an effective memory interface across diverse memory demands without an observed success-rate trade-off on standard manipulation.

**RMBench.** Table 1 compares SimpleMemVLA with all published baselines on the nine RMBench tasks for which prior results are available, where every baseline is trained as a separate task-specific specialist. Using a single model across all ten tasks, SimpleMemVLA achieves 94.0% on the nine-task comparison set, 11.0 points above the strongest specialist. More importantly, among the methods in Table 1, SimpleMemVLA is the only one whose average increases from the single-memory setting to the multi-memory setting, from 91.6% to 97.0%. This result suggests that using native

Table 1: **RMBench** per-task success rates (%), averaged over the nine tasks with published baselines. Every baseline trains one specialist model per task while SimpleMemVLA is a single multi-task model evaluated at n=100 seeds per task in the streaming deployment of Section 4.5. †No prior VLA reports Place-Mat so it is excluded from all averages. Best per column in **bold**.

| | Single-memory $M(1)$ | | | | | | Multi-memory $M(n)$ | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Method | Obs&PU | Rearr. | PutBack | SwapB | SwapT | *Avg* | Battery | Rank. | Cover | Press | Place-Mat† | *Avg* | Overall |
| *One specialist model per task* | | | | | | | | | | | | | |
| DP (Chi et al., 2023) | 1 | 0 | 0 | 11 | 20 | 6.4 | 10 | 10 | 0 | 0 | – | 5.0 | 5.8 |
| ACT (Zhao et al., 2023) | 1 | 29 | 0 | 2 | 2 | 6.8 | 19 | 0 | 0 | 0 | – | 4.8 | 5.9 |
| $\pi_{0.5}$ (Intelligence et al., 2025) | 9 | 13 | 11 | 24 | 15 | 14.4 | 16 | 6 | 0 | 0 | – | 5.5 | 10.4 |
| X-VLA (Zheng et al., 2026) | 9 | 13 | 18 | 16 | 3 | 11.8 | 26 | 1 | 2 | 0 | – | 7.3 | 9.8 |
| Mem-0 (Chen et al., 2026) | 4 | 89 | 90 | 67 | 14 | 52.8 | 28 | 18 | 68 | 0 | – | 28.5 | 42.0 |
| HiMem-WAM (Sun et al., 2026b) | 28 | 33 | 32 | 38 | 27 | 31.6 | 28 | 24 | 19 | 8 | – | 19.8 | 26.3 |
| EventVLA (Yang et al., 2026a) | 21 | 96 | 95 | 96 | 87 | 79.0 | 35 | 81 | 97 | 3 | – | 54.0 | 67.8 |
| DIM-WAM (Wang et al., 2026a) | 13 | 99 | 98 | 96 | **97** | 80.6 | 48 | 87 | 56 | 34 | – | 56.3 | 69.8 |
| MemoryWAM (Yang et al., 2026b) | 27 | **100** | **100** | **100** | 94 | 84.2 | 41 | **100** | **98** | 87 | – | 81.5 | 83.0 |
| *Single multi-task checkpoint* | | | | | | | | | | | | | |
| SimpleMemVLA (Ours) | **65** | **100** | **100** | **100** | 93 | **91.6** | **90** | **100** | **98** | **100** | **100** | **97.0** | **94.0** |

context as memory allows the policy to revisit the specific event needed for the current action as more past events become relevant.

**RoboMME and MIKASA-Robo.** On the sixteen-task RoboMME suite, a single model reaches 88.3% (Table 2), 43.7 points above the strongest non-oracle baseline, first on all sixteen tasks (Figures 8, 9 and 10) and above the GroundSG reference supplied with ground-truth perception (84.1%), a gap Appendix D.2 traces to write time. MIKASA-Robo shows and removes its cues before the robot may act, so the decisive evidence exists only in the history: SimpleMemVLA reaches 74.0% (Table 3), 29.6 points above the strongest prior VLA and 6.2 points above GMP, a memory-specialized non-VLA policy trained separately per task. Across both suites the lead covers every memory dimension, from event counts and hidden-object states to referenced observations and demonstrated sequences, and Appendix D.1 shows the gap widening with memory demand.

**RoboMemArena.** RoboMemArena tests whether this advantage persists over substantially longer interactions, with twenty-six tasks whose trajectories average more than one thousand control steps. With one model and a 126 s native context window, SimpleMemVLA reaches 63.6% TSR and 72.1% CSR (Table 4), +17.4 points over the strongest prior model and above the benchmark's own ground-truth reference (46.1%), the ceiling of its dual-system architecture. The category breakdown shows where the overall gain comes from. Compared with FrameSamp+Modul, SimpleMemVLA increases TSR from 39.1% to 64.3% on Occlusion, where the robot must remember objects after they disappear, and from 31.4% to 71.4% on Counting, where it must track completed events throughout the episode. It does not lead on Transferring. The largest improvements therefore occur on tasks that explicitly require information from earlier observations, indicating that native context improves memory-dependent behavior rather than low-level control in general.

**LIBERO and LIBERO-Plus.** The two general-purpose suites serve as the control: they test whether exposing a minute-scale visual history compromises standard, largely reactive manipulation. SimpleMemVLA reaches 97.5% across the four LIBERO suites (Table 5), tying the best reported average, and the same model transfers zero-shot to the 10,030 perturbed LIBERO-Plus tasks at 78.4% (Table 6), 5.3 points above the strongest memory-augmented model. RIPT-VLA ties SimpleMemVLA on standard LIBERO yet drops 29.1 points under perturbation against 19.1 for SimpleMemVLA, consistent with the retained history serving as a stable visual reference (Appendix C).

### 4.3 Attribution: Isolating the Memory Interface

The gains of Section 4.2 admit a competing explanation: they could come from a stronger backbone rather than from the memory interface itself. To rule this out, we rebuild one representative method

Table 2: **RoboMME** category-level success rates (%). AVG is over all sixteen tasks. Gray rows are reference-only and excluded from ranking. **Bold** and underline mark ranks 1 and 2. The SimpleMemVLA variant rows re-create one mechanism family each on the otherwise unchanged SimpleMemVLA stack. Per-task results are in Table 8.

| Memory | Method | Integr. / VLM | Counting | Permanence | Reference | Imitation | AVG |
|---|---|---|---|---|---|---|---|
| Human | Human Performance | - | 88.5 | 91 | 93 | 89.5 | 90.5 |
| Symbolic (Oracle) | SimpleSG (Dai et al., 2026) | GT VLM | 82.6 | 21.6 | 32.3 | 61.9 | 49.6 |
| Symbolic (Oracle) | GroundSG (Dai et al., 2026) | GT VLM | 83.9 | 93.3 | 95.2 | 64.0 | 84.1 |
| Symbolic | SimpleSG | Gemini | 39.0 | 13.5 | 22.5 | 18.0 | 23.3 |
| Symbolic | SimpleSG | QwenVL | 44.6 | 19.6 | 25.2 | 26.6 | 29.0 |
| Symbolic | GroundSG | Gemini | 12.8 | 15.8 | 11.3 | 6.5 | 11.6 |
| Symbolic | GroundSG | QwenVL | 38.0 | 39.3 | 31.6 | 21.9 | 32.7 |
| Perceptual | TokenDrop (Dai et al., 2026) | Context | 57.9 | 26.9 | 23.7 | 29.5 | 34.5 |
| Perceptual | TokenDrop | Modul | 52.3 | 26.8 | 34.7 | 38.3 | 38.0 |
| Perceptual | TokenDrop | Expert | 59.4 | 23.6 | 25.3 | 31.1 | 34.9 |
| Perceptual | FrameSamp (Dai et al., 2026) | Context | 50.1 | 23.3 | 20.9 | 28.3 | 30.7 |
| Perceptual | FrameSamp | Modul | 65.2 | 25.1 | 36.3 | <u>51.4</u> | <u>44.5</u> |
| Perceptual | FrameSamp | Expert | <u>66.8</u> | 25.2 | 24.1 | 28.9 | 36.3 |
| Recurrent | TTT (Dai et al., 2026) | Context | 36.1 | 21.4 | 22.0 | 9.7 | 22.3 |
| Recurrent | TTT | Modul | 34.5 | 22.2 | 20.4 | 10.7 | 22.0 |
| Recurrent | TTT | Expert | 36.0 | 22.9 | 19.7 | 10.8 | 22.3 |
| Recurrent | RMT (Dai et al., 2026) | Context | 32.2 | 15.4 | 19.7 | 10.6 | 19.5 |
| Recurrent | RMT | Modul | 34.1 | 15.8 | 21.1 | 9.7 | 20.2 |
| Recurrent | RMT | Expert | 34.4 | 15.7 | 14.7 | 7.8 | 18.1 |
| Other | $\pi_{0.5}$ (Intelligence et al., 2025) | - | 28.8 | 17.0 | 17.2 | 8.8 | 17.9 |
| Other | WeaveLA (Zhu et al., 2026) | - | 31.0 | 19.5 | 12.5 | 9.5 | 18.1 |
| Other | $\pi_{0.5}$ w/ past actions (Dai et al., 2026) | - | 29.1 | 22.8 | 15.9 | 11.2 | 19.7 |
| Other | SAM2Act+ (Fang et al., 2025) | - | 35.3 | 26.0 | 16.8 | 7.3 | 21.4 |
| Other | MemER (Sridhar et al., 2026) | - | 48.8 | <u>53.2</u> | <u>38.0</u> | 29.5 | 42.4 |
| Ours variant | SimpleMemVLA (Retrieval) | - | 46.5 | 42.0 | 13.5 | 24.0 | 31.5 |
| Ours variant | SimpleMemVLA (Token Compression) | - | 46.0 | 13.5 | 12.5 | 18.5 | 22.6 |
| Ours variant | SimpleMemVLA (Recurrent State) | - | 31.0 | 24.0 | 17.5 | 10.0 | 20.6 |
| Ours | SimpleMemVLA | - | **91.5** | **96.0** | **82.5** | **83.0** | **88.3** |

Table 3: **MIKASA-Robo** five-task success rates (%). GMP is a per-task non-VLA reference. Best VLA per column in **bold**.

| Method | ShellGame | Intercept | RC-3 | RC-5 | RC-9 | Avg |
|---|---|---|---|---|---|---|
| *Vision-language-action models* | | | | | | |
| CronusVLA (Li et al., 2026) | 32 | 5 | 31 | 13 | 9 | 18.0 |
| SpatialVLA (Qu et al., 2025) | 23 | 27 | 27 | 17 | 11 | 21.0 |
| OpenVLA-OFT (Kim et al., 2025) | 47 | 14 | 59 | 16 | 6 | 28.4 |
| $\pi_0$ (Black et al., 2025) | 33 | 42 | 35 | 22 | 15 | 29.4 |
| Octo (Team et al., 2024) | 46 | 39 | 45 | 17 | 11 | 31.6 |
| MemoryVLA (Shi et al., 2026a) | 88 | 24 | 44 | 30 | 20 | 41.2 |
| MemoryVLA++ (Shi et al., 2026b) | 97 | 40 | 50 | 19 | 16 | 44.4 |
| *Compact non-VLA memory policy (reference)* | | | | | | |
| GMP (Gao et al., 2026) | 98 | 83 | **80** | **61** | 17 | 67.8 |
| SimpleMemVLA (Ours) | **99** | **83** | 71 | 58 | **59** | **74.0** |

from each mechanism family on the otherwise unchanged stack, holding the training data, backbone, sub-task supervision, action head and optimizer fixed, so the variants differ only in how historical observations are exposed to the backbone. Under this matched setting, native context reaches 88.3% on RoboMME, whereas the retrieval, token-compression and recurrent-state variants reach 31.5%, 22.6% and 20.6% (Table 2).

Table 4: **RoboMemArena** category-level TSR and CSR (%) over all 26 tasks under the official protocol. MemER is the benchmark authors' reimplementation, FrameSamp+Modul the only external leaderboard entry, the gray oracle row excluded from ranking. Best per column in **bold**.

| | Transferring | | Occlusion | | Counting | | Sequence | | Average | |
|---|---|---|---|---|---|---|---|---|---|---|
| Method | TSR | CSR | TSR | CSR | TSR | CSR | TSR | CSR | TSR | CSR |
| $\pi_{0.5}$ (Intelligence et al., 2025) | 20.0 | 42.8 | 12.7 | 17.2 | 14.3 | 50.9 | 60.0 | 71.6 | 21.5 | 38.7 |
| HiF-VLA (Lin et al., 2025b) | 17.5 | 38.9 | 12.7 | 27.1 | 8.6 | 45.9 | 42.5 | 70.2 | 16.9 | 39.8 |
| MemoryVLA (Shi et al., 2026a) | 15.0 | 37.2 | 7.3 | 13.1 | 14.3 | 55.1 | 37.5 | 65.2 | 15.0 | 35.3 |
| MemER (Sridhar et al., 2026) | 20.0 | 36.1 | 16.4 | 33.2 | 27.1 | 65.1 | 65.0 | 79.1 | 27.3 | 49.1 |
| FrameSamp+Modul (Dai et al., 2026) | **63.8** | **72.1** | 39.1 | 56.5 | 31.4 | 57.8 | 73.8 | 86.9 | 46.2 | 63.9 |
| PrediMem (Lei et al., 2026) | 22.5 | 45.2 | 27.3 | 38.4 | 45.7 | 69.3 | 72.5 | **89.5** | 38.5 | 55.2 |
| Ground truth (oracle) (Lei et al., 2026) | 32.5 | 54.8 | 33.6 | 49.8 | 51.4 | 75.6 | 85.0 | 92.3 | 46.1 | 64.8 |
| SimpleMemVLA (Ours) | 35.8 | 37.4 | **64.3** | **76.8** | **71.4** | **79.3** | **75.5** | 81.1 | **63.6** | **72.1** |

Table 5: **LIBERO** success rates (%) on the four standard suites, 500 trials per suite. Best per column in **bold**.

| Method | Spatial | Object | Goal | Long | Avg |
|---|---|---|---|---|---|
| *General-purpose policies and VLAs* | | | | | |
| Diffusion Policy (Chi et al., 2023) | 78.3 | 92.5 | 68.3 | 50.5 | 72.4 |
| TraceVLA (Zheng et al., 2025) | 84.6 | 85.2 | 75.1 | 54.1 | 74.8 |
| Octo (Team et al., 2024) | 78.9 | 85.7 | 84.6 | 51.1 | 75.1 |
| OpenVLA (Kim et al., 2024) | 84.7 | 88.4 | 79.2 | 53.7 | 76.5 |
| SpatialVLA (Qu et al., 2025) | 88.2 | 89.9 | 78.6 | 55.5 | 78.1 |
| $\pi_0$-FAST (Pertsch et al., 2025) | 96.4 | 96.8 | 88.6 | 60.2 | 85.5 |
| $\pi_0$ (Black et al., 2025) | 96.8 | 98.8 | 95.8 | 85.2 | 94.2 |
| DeepThinkVLA (Yin et al., 2026) | 96.6 | 99.0 | 96.4 | **96.2** | 97.0 |
| OpenVLA-OFT (Kim et al., 2025) | 97.6 | 98.4 | 97.9 | 94.5 | 97.1 |
| RIPT-VLA (Tan et al., 2025) | – | – | – | – | **97.5** |
| *Memory-augmented VLAs* | | | | | |
| MemoryVLA (Shi et al., 2026a) | **98.4** | 98.4 | 96.4 | 93.4 | 96.7 |
| CronusVLA (Li et al., 2026) | 97.3 | **99.6** | 96.9 | 94.0 | 97.0 |
| ContextVLA (Jang et al., 2025) | **98.4** | 99.0 | 97.2 | 93.4 | 97.0 |
| SimpleMemVLA (Ours) | 98.2 | 98.8 | **98.0** | 95.0 | **97.5** |

The key distinction is how much of the original history remains available when the policy makes the current decision. The retrieval variant exposes eight uniformly sampled frames as unordered images without timestamps; token compression maps the full history into a fixed 64-token representation using queries that are independent of the current decision; and recurrent state continually rewrites the past into a fixed 16-token representation. Each alternative therefore restricts what the backbone can read before its task-conditioned attention determines which perceptual and temporal evidence is needed. SimpleMemVLA instead preserves the sampled visual observations, their temporal order and their plaintext timestamps, deferring evidence selection to the backbone's native self-attention at decision time.

The task-level results reveal the different consequences of these bottlenecks (Figure 4). Retrieval performs best among the three alternatives because it retains raw visual content: it is the only variant that remains effective on Permanence (42.0%), where the answer is a specific past percept. However, presenting the retrieved frames without order or timestamps reduces the swap and ordered-pattern tasks to 10–20%, showing that visual content alone is insufficient when an observation must be interpreted by when it occurred. Token compression observes every history frame, but Counting remains its strongest dimension at only 46.0%, while its overall performance falls to 22.6%. This pattern is consistent with a fixed token budget retaining coarse aggregate information while discarding the task-specific perceptual details needed later. The recurrent variant reaches 20.6% overall and is strongest on Counting at 31.0%, consistent with its fixed state retaining some running aggregate information. Performance falls to 24.0% on Permanence, 17.5% on Reference and 10.0% on Imitation, indicating that the state does not reliably preserve the task-specific perceptual and temporal

Table 6: **LIBERO-Plus** zero-shot robustness transfer to the 10,030 perturbed tasks, all policies trained on standard LIBERO only. Best per column in **bold**.

| Method | Camera | Robot | Lang. | Light | Backg. | Noise | Layout | Total |
|---|---|---|---|---|---|---|---|---|
| *General-purpose policies and VLAs* | | | | | | | | |
| OpenVLA (Kim et al., 2024) | 0.8 | 3.5 | 23.0 | 8.1 | 34.8 | 15.2 | 28.5 | 15.6 |
| WorldVLA (Cen et al., 2025) | 0.1 | 27.9 | 41.6 | 43.7 | 17.1 | 10.9 | 38.0 | 25.0 |
| Diffusion Policy (Chi et al., 2023) | 1.6 | 32.3 | 77.0 | 25.0 | 19.8 | 20.3 | 42.2 | 31.7 |
| NORA (Hung et al., 2025) | 2.2 | 37.0 | 65.1 | 45.7 | 58.6 | 12.8 | 62.1 | 39.0 |
| UniVLA (Bu et al., 2025) | 1.8 | 46.2 | 69.6 | 69.0 | 81.0 | 21.2 | 31.9 | 42.9 |
| Fast-WAM (Yuan et al., 2026) | 16.4 | 44.5 | 68.9 | 78.2 | 53.7 | 37.7 | 60.7 | 51.5 |
| $\pi_0$ (Black et al., 2025) | 13.8 | 6.0 | 58.8 | 85.0 | 81.4 | **79.0** | 68.9 | 53.6 |
| $\pi_0$-FAST (Pertsch et al., 2025) | 65.1 | 21.6 | 61.0 | 73.2 | 73.2 | 74.4 | 68.8 | 61.6 |
| OpenVLA-OFT$_m$ (Kim et al., 2025) | 55.6 | 21.7 | 81.0 | 92.7 | 91.0 | 78.6 | 68.7 | 67.9 |
| RIPT-VLA (Tan et al., 2025) | 55.2 | 31.2 | 77.6 | 88.4 | 91.6 | 73.5 | 74.2 | 68.4 |
| OpenVLA-OFT (Kim et al., 2025) | 56.4 | 31.9 | 79.5 | 88.7 | 93.3 | 75.8 | 74.2 | 69.6 |
| *Memory-augmented VLAs* | | | | | | | | |
| MemoryVLA (Shi et al., 2026a) | 42.7 | 44.9 | 84.4 | 92.8 | 95.0 | 62.1 | **84.7** | 70.2 |
| MemoryVLA++ (Shi et al., 2026b) | 36.4 | **68.9** | **88.7** | 93.8 | 90.6 | 63.5 | 83.8 | 73.1 |
| SimpleMemVLA (Ours) | **76.9** | 68.4 | 68.6 | **96.4** | **96.8** | 74.2 | 78.0 | **78.4** |

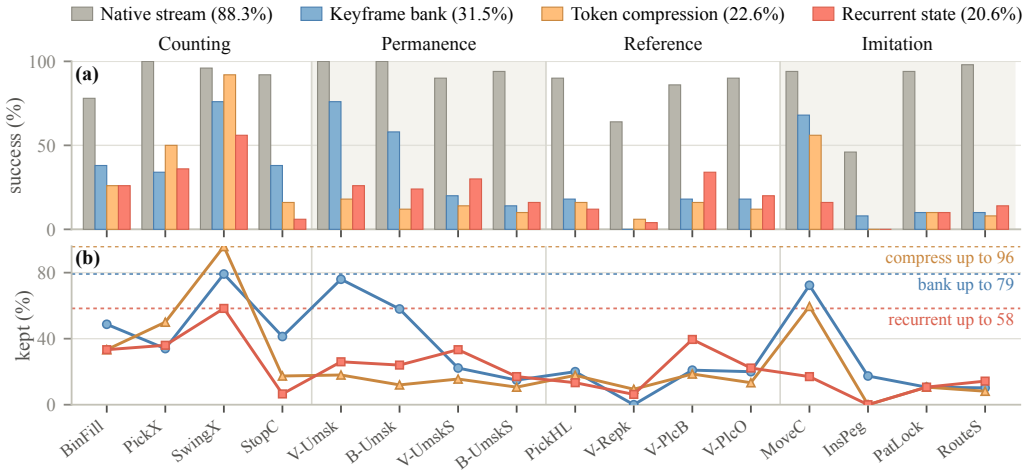



Figure 4: **Task-level effects of restricted memory interfaces on RoboMME.** The three controlled variants differ from SimpleMemVLA only in how history enters the model. *(a)* Success rates across 16 tasks grouped by benchmark dimension. *(b)* The same results normalized to native-context performance. Retrieval, token compression and recurrent state retain at most 79%, 96% and 58%, respectively, with the token-compression peak confined to the count task SwingXtimes. Results are from Table 8, with $n = 50$ episodes per task.

evidence required across the suite. In this implementation, the recurrent state must determine what to retain at each step, even though the information required by future decisions may not yet be known. Because training backpropagates through only four decisions, supervision from later decisions cannot directly teach earlier state updates which observations should have been preserved (Williams & Peng, 1990; Bulatov et al., 2022).

The three matched alternatives impose different restrictions before the current task-conditioned read: retrieval limits which frames and temporal metadata reach the backbone, token compression limits representational capacity, and recurrent state repeatedly folds the past into a fixed-size representation. Native context avoids this pre-read bottleneck by preserving both perceptual detail and temporal structure until read time. In this tested setting, these controlled results support a read-time view of memory: preserving the history until the current decision allows the backbone itself to determine which past observations matter, rather than requiring a restricted memory interface to decide in ad-

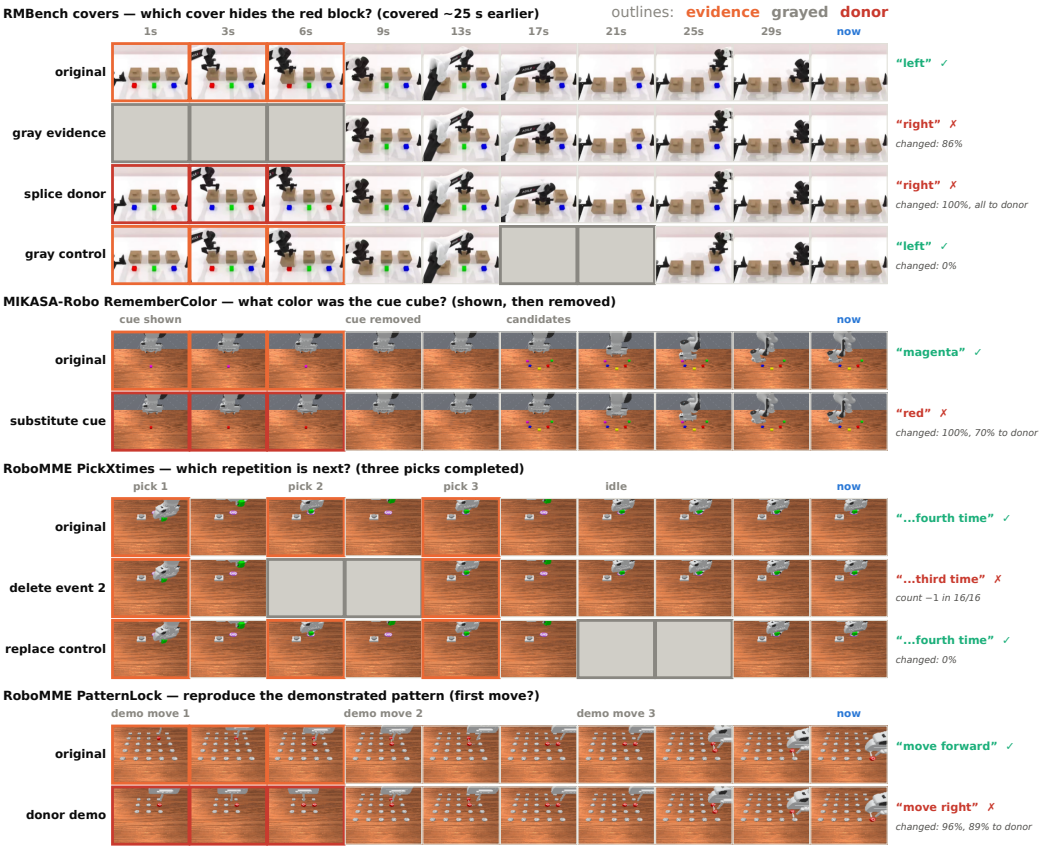



Figure 5: **History interventions redirect the output across all suites.** Blocks denote benchmarks and rows show histories from oldest to most recent, processed through the unchanged deployment pipeline. Orange borders mark evidence frames, gray fills evidence ablations and red borders donor-episode insertions. Removing evidence changes the output, donor evidence redirects it toward the donor content and matched control edits leave it unchanged.

vance what to retain. MEM (Torne et al., 2026) supplies an external data point from the text side: carrying history as the raw sequence of past sub-task instructions performs significantly worse than its compressed summaries, because deployed policies repeat failed sub-tasks and produce text histories that training never contains. A textual past therefore chooses between distribution shift and a write-time decision about what to discard, a dilemma the sampled video stream never enters because it is the same object at training and deployment. Complete task-level results and additional analysis are provided in Appendix B and Appendix D.2.

### 4.4 Genuine Memory: Causal History Use and Emergent Visual In-Context Learning

The benchmark results establish that SimpleMemVLA performs well on memory-dependent tasks. We next test whether the deployed policy actually uses its visual history as memory. At each target decision, we hold the model, instruction, current observation, robot state and deployment path fixed, modify only the historical video, and regenerate the policy output.

**The policy genuinely uses its visual memory.** Masking the history affects the policy specifically when the removed frames contain evidence required by the current decision. On the RMBench cover-blocks task, masking the historical video segment that records where the red block was covered causes the policy to select the wrong cover, whereas masking task-irrelevant history leaves its output unchanged (Figure 5 row 2 and 4). On MIKASA-Robo RememberColor, masking the historical frames that contain the color cue causes the policy to select the wrong color, whereas

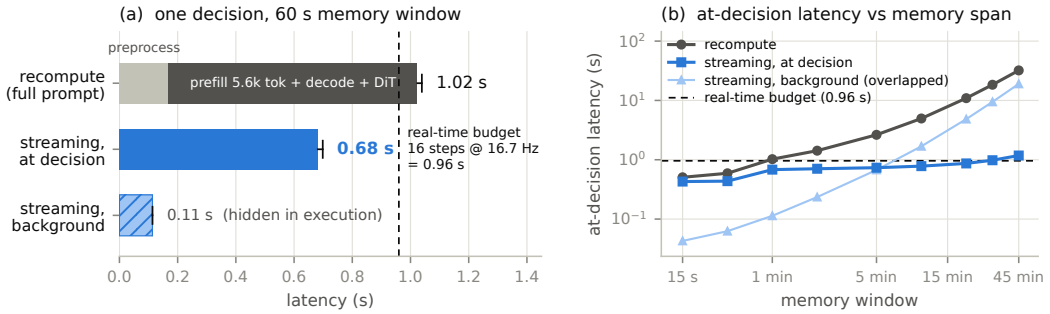



Figure 6: **Streaming keeps at-decision latency near the single-frame cost across 15 s–45 min histories.** All measurements use unique frames on one H100 with bf16 and batch size 1. *(a)* At 60 s, streaming reduces latency from 1.02 s to 0.68 s, below the 0.96 s real-time budget. *(b)* At 45 min (245k tokens), full recomputation reaches 32.1 s while streaming requires only 1.18 s. Both paths produce byte-identical sub-task text.

masking task-irrelevant frames leaves its output unchanged. Together with the RMBench result, this shows that the policy fails specifically when task-relevant visual memory is removed, rather than in response to visual masking itself.

RoboMME PickXtimes provides a more fine-grained test. Masking one completed pick-and-place event causes the policy to infer that one fewer repetition has occurred—for example, changing its output from "the fourth time" to "the third time"—and therefore to execute one additional pick. Masking a matched history segment that contains no pick-and-place event leaves the inferred count unchanged. The policy therefore remembers the exact number of completed events, rather than merely whether a relevant event has occurred (Figure 5 row 8 and 9).

**Emergent visual in-context learning.** On RMBench, replacing the original covering event with frames from another rollout in which the red block is placed under a different cover causes the policy to select the cover shown in the replacement video. Replacing it with frames showing the same cover leaves the decision unchanged (Figure 5 row 3). On MIKASA-Robo, replacing a magenta cue with a red cue causes the policy to grasp the red block (Figure 5 row 5). On RoboMME PatternLock, replacing a demonstration beginning with "move forward" by one beginning with "move right" causes the policy to execute "move right," whereas replacing it with another demonstration of the same first move leaves the behavior unchanged (Figure 5 row 11).

Neither type of edited history appears during training. The replacement conditions splice a visual segment from one rollout into the remaining history of another, while the PickXtimes condition masks one completed event in an otherwise valid rollout. Without intervention-specific supervision or parameter updates, the frozen policy nevertheless interprets these newly constructed visual histories at inference time: it follows the substituted location, color, and demonstration, and adjusts its count to the completed events that remain visible.

This behavior reveals an emergent visual in-context learning capability. The frozen policy can infer the task state or intended behavior directly from a newly constructed visual context and adapt its current action accordingly. Together, the masking and replacement experiments show that native context serves both as genuine memory of past events and as an inference-time visual interface through which new evidence can re-specify the policy's behavior.

### 4.5 Efficiency: Streaming Inference at Single-Frame-Scale Decision Cost

The preceding results establish both the effectiveness of SimpleMemVLA and its genuine use of visual history. The remaining practical question is the cost of processing that history. With a minute-scale context window, full recomputation processes a prompt of at least 5k tokens at every decision, compared with approximately 0.5k tokens for a single-frame VLA.

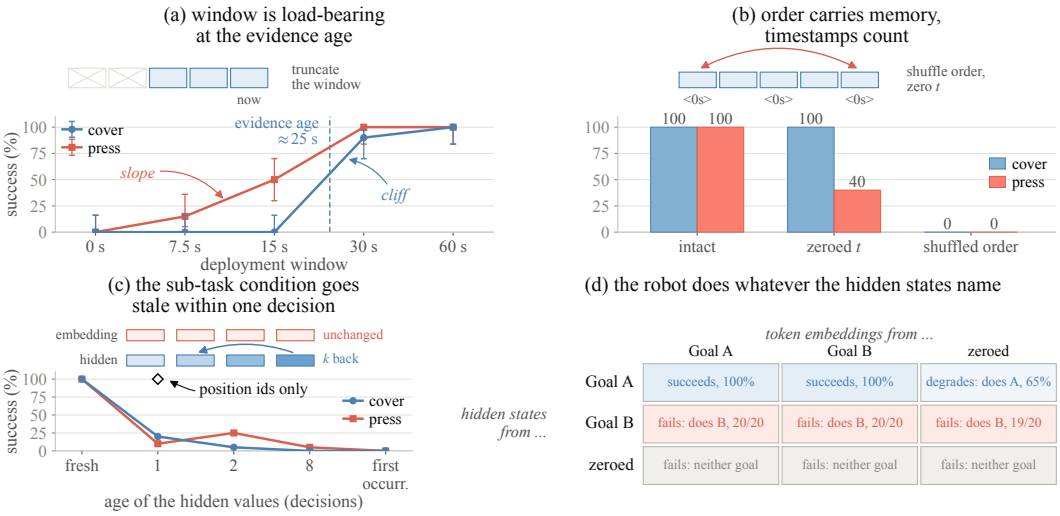



Figure 7: **Native-context memory relies on retained temporal evidence and contextual hidden states.** *(a)* Cover-blocks fails once the covering event leaves the window, whereas press-button degrades gradually as completed presses are removed. *(b)* Both tasks depend on frame order, while timestamps matter primarily for counting. *(c)* Stale sub-task hidden states sharply reduce success, whereas equally stale position IDs leave it unchanged. *(d)* Behavior follows the source of the hidden states rather than the token embeddings, identifying contextual hidden states as the memory-to-action interface.

The exact streaming implementation of Section 3.4 moves this recomputation off the decision-critical path (Figure 2b) while producing the same generated sub-task.

On one H100, for the representative minute-scale configuration evaluated in Figure 6a, streaming reduces decision latency from 1.02 s to 0.68 s. Its additional 0.11 s of background prefill fits within the 0.96 s action-execution interval and therefore does not extend the decision-critical path.

The advantage increases with context length. In our scaling experiment, 45 min of history corresponds to approximately 245k tokens, near the backbone's 262k position limit. At this same 45 min point, full-recomputation latency reaches 32.1 s per decision, whereas the streamed decision path requires only 1.18 s (Figure 6b).

The background prefill and key–value cache of exact streaming nevertheless continue to grow with the accumulated history, eventually limiting the sustainable decision rate. This remaining dependence on episode length can be removed by training the policy for fully streaming deployment. Appendix E presents a sliding-window-attention(SWA) variant that uses the same bounded attention window during training and inference. At deployment, it appends one new temporal patch and evicts one expired cache block at every decision, keeping both background prefill and cache state independent of the total episode length (Figure 11).

Exact streaming therefore brings minute-scale context to single-frame-scale decision latency, while streaming-native SWA extends bounded prefill and cache costs to substantially longer episodes.

### 4.6 Ablations: Dissecting the Memory Pathway

We ablate the key components of native context as memory to determine the role of each: the history window, frame order, plaintext timestamps, and the hidden-state interface between the generated sub-task and the action head. We conduct these ablations on two RMBench tasks with complementary memory requirements. Cover-blocks depends on a single covering event about 25 s earlier, whereas press-button requires tracking completed presses throughout the trajectory (Figure 7). **The evidence must remain in the window.** On cover-blocks, a 30 s history still contains the moment when the red block is covered, whereas a 15 s history removes this evidence and causes the task to fail. On press-button, shortening the window progressively removes previously completed presses,

making it increasingly difficult for the policy to determine the correct count. With only the current frame, the policy can observe neither where the block was covered nor how many presses have been completed, and both tasks fail (Figure 7a). **Order and timestamps provide complementary temporal information.** Shuffling frame order causes both tasks to fail (Figure 7b). Removing plaintext timestamps leaves cover-blocks unaffected but substantially degrades press-button. Frame order supports remembering a single event, while counting distributed events additionally relies on explicit temporal information. **Hidden states carry visual memory into action.** The generated sub-task reaches the action head through final-layer hidden states and token embeddings. Replacing the embeddings with those of another target leaves the selected behavior nearly unchanged, whereas replacing or zeroing the hidden states collapses task success and can redirect the policy toward the target encoded by the substituted states (Figure 7d). The history-dependent control signal is therefore carried primarily by the hidden states. The embeddings support stable execution, but their identity does not determine the selected target. **The memory is refreshed at every decision.** Deleting the target word makes the visible sub-task identical across decisions, yet the policy still performs the seed-specific cover sequence and press count. This information must therefore remain in the contextual hidden states rather than in the text alone. Reusing hidden states from even one earlier decision sharply degrades both tasks, while equally stale position identifiers do not (Figure 7c). The generated sub-task span consequently acts as a working memory: at each decision, the backbone reads the visual history and writes the current task state into its hidden representation, which the action head then uses to produce the next action.

## 5 Conclusion

Our results show that a pretrained VLM's native video context can serve as an effective memory interface for long-horizon VLA control without a dedicated memory module. SimpleMemVLA exposes minute-scale timestamped visual history directly to the backbone and conveys the resulting task state to a standard flow-matching action head through the hidden states of a generated sub-task. This recipe alone establishes new state-of-the-art results across four memory-centric benchmarks without sacrificing performance on the evaluated general-purpose suites. A matched comparison attributes the gain to the memory interface itself: on RoboMME, native context reaches 88.3% where retrieval, token-compression and recurrent-state rebuilds of the same stack reach 31.5%, 22.6% and 20.6%. These results support preserving perceptual detail and temporal structure until task-conditioned read time rather than restricting history in advance. History-editing experiments confirm that the policy causally depends on specific past observations and reveal emergent visual in-context learning: previously unseen edited histories redirect the output at inference time without parameter updates. Ablations assign each element its role: the evidence must stay within the history window, shuffling frame order or removing plaintext timestamps breaks the temporal reading and the sub-task hidden states must be rewritten at every decision, acting as a working memory between the visual history and the action head. Exact streaming makes this memory practical at deployment, reducing decision latency from 1.02 s to 0.68 s at the minute scale while producing the same generated sub-task as full recomputation. Although our findings are limited to the evaluated backbones, benchmarks and memory interfaces, we argue that native context should serve as a matched baseline for future VLA memory mechanisms. Before adding dedicated memory machinery, the first comparison should be the same policy given access to its own visual history.

## References


Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. In *Neural Information Processing Systems*, 2022. doi: 10.52202/068431-1723.

Shuai Bai, Yuxuan Cai, Rui-Zhe Chen, Ke-qin Chen, Xiong-Hui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Rongyao Fang, Chang Gao, et al. Qwen3-VL technical report. *arXiv.org*, 2025a. doi: 10.48550/arXiv.2511.21631.

Shuai Bai, Ke-qin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, K. Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-VL technical report. *arXiv.org*, 2025b. doi: 10.48550/arXiv.2502.13923.

Kevin Black, Noah Brown, Danny Driess, A. Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. $\pi_0$: A vision-language-action flow model for general robot control. *Robotics: Science and Systems XXI*, 2025. doi: 10.15607/rss.2025.xxi.010.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, K. Choromanski, Tianli Ding, Danny Driess, Kumar Avinava Dubey, Chelsea Finn, Peter R. Florence, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, 2023. doi: 10.48550/arXiv.2307.15818.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, J. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, G. Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Neural Information Processing Systems*, 2020.

Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. UniVLA: Learning to act anywhere with task-centric latent actions. In *Robotics: Science and Systems XXI*, 2025. doi: 10.15607/rss.2025.xxi.014.

Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. In *Advances in Neural Information Processing Systems 35*, 2022. doi: 10.52202/068431-0805.

Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, et al. WorldVLA: Towards autoregressive action world model. *arXiv.org*, 2025. doi: 10.48550/arXiv.2506.21539.

Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, et al. GR-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv.org*, 2024. doi: 10.48550/arXiv.2410.06158.

Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Qiwei Liang, Zixuan Li, Xianlian Lin, Yiheng Ge, Zhenyu Gu, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv.org*, 2025. doi: 10.48550/arXiv.2506.18088.

Tianxing Chen, Yuran Wang, Mingleyang Li, Yanjiao Qin, Haochen Shi, Zixuan Li, Yifan Hu, Yingsheng J Zhang, Kaixuan Wang, Yue Chen, et al. Rmbench: Memory-dependent robotic manipulation benchmark with insights into policy design. *arXiv.org*, 2026. doi: 10.48550/arxiv.2603.01229.

E. Cherepanov, Nikita Kachaev, A. Kovalev, and Aleksandr I. Panov. Memory, benchmark & robots: A benchmark for solving complex tasks with reinforcement learning. *arXiv.org*, 2025. doi: 10.48550/arXiv.2502.10550.

Egor Cherepanov, Nikita Kachaev, Daniil Zelezetsky, Aydar Bulatov, Artem Pshenitsyn, Yuri Kuratov, Alexey Skrynnik, Aleksandr I. Panov, and Alexey K. Kovalev. $\mu$vla: On recurrent memory for partially observable manipulation in vla models, 2026.

Cheng Chi, S. Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, B. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems Conference*, 2023. doi: 10.1177/02783649241273668.

Yinpei Dai, H. Fu, Jayjun Lee, Yuejiang Liu, Haoran Zhang, Jianing Yang, Chelsea Finn, Nima Fazeli, and Joyce Chai. RoboMME: Benchmarking and understanding memory for robotic generalist policies. In *arXiv.org*, 2026. doi: 10.48550/arXiv.2603.04639.

Haoquan Fang, Markus Grotz, Wilbert Pumacay, Yi Ru Wang, Dieter Fox, Ranjay Krishna, and Jiafei Duan. SAM2Act: Integrating visual foundation model with a memory architecture for robotic manipulation. In *International Conference on Machine Learning*, 2025. doi: 10.48550/arXiv.2501.18564.

Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.

Yihuai Gao, Jeff Jinyun Liu, Shuang Li, and Shuran Song. Gated memory policy: In-context memorization and adaptation. *arXiv*, 2026. doi: 10.48550/arxiv.2604.18933.

Yanjiang Guo, L. Shi, Jianyu Chen, and Chelsea Finn. Ctrl-World: A controllable generative world model for robot manipulation. *arXiv.org*, 2025. doi: 10.48550/arXiv.2510.10125.

Yanbin Hu, Jin Cui, Jiayi Lu, Ruixuan Yang, Junjie Ye, Bo Zhao, Xingyu Chen, Xuguang Lan, and Pengju Ren. ECHO: Continuous hierarchical memory for vision-language-action models. *arXiv.org*, 2026. doi: 10.48550/arXiv.2605.10993.

Yuzhi Huang, Weijue Bu, Ziyi Xiong, Jie Wu, Fanding Huang, Jingyan Jiang, and Zhi Wang. Chain-vla: Chaining vision-language-action queries through a unified execution state for long-horizon manipulation. *arXiv preprint arXiv:2608.02326*, 2026.

Chia-Yu Hung, Qi Sun, Pengfei Hong, Amir Zadeh, Chuan Li, U-Xuan Tan, Navonil Majumder, and Soujanya Poria. NORA: A small open-sourced generalist vision language action model for embodied tasks. *arXiv.org*, 2025. doi: 10.48550/arXiv.2504.19854.

Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, A. Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv.org*, 2025. doi: 10.48550/arXiv.2504.16054.

Huiwon Jang, Sihyun Yu, Heeseung Kwon, Hojin Jeon, Younggyo Seo, and Jinwoo Shin. ContextVLA: Vision-language-action model with amortized multi-frame context. *arXiv.org*, 2025. doi: 10.48550/arXiv.2510.04246.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 1998. doi: 10.1016/S0004-3702(98)00023-X.

Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, A. Balakrishna, Suraj Nair, Rafael Rafailov, E. Foster, Grace Lam, Pannag R. Sanketi, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, 2024. doi: 10.48550/arXiv.2406.09246.

Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. In *Robotics: Science and Systems XXI*, 2025. doi: 10.15607/rss.2025.xxi.017.

Myungkyu Koo, Daewon Choi, Taeyoung Kim, Kyungmin Lee, Changyeon Kim, Y. Seo, and Jinwoo Shin. HAMLET: Switch your vision-language-action model into a history-aware policy. *arXiv.org*, 2025. doi: 10.48550/arXiv.2510.00695.

Huashuo Lei, Wenxuan Song, Huarui Zhang, Jieyuan Pei, Jiayi Chen, Haodong Yan, Han Zhao, Pengxiang Ding, Zhipeng Zhang, Lida Huang, et al. RoboMemArena: A comprehensive and challenging robotic memory benchmark. *arXiv.org*, 2026. doi: 10.48550/arXiv.2605.10921.

Mingcong Lei, Honghao Cai, Binbin Que, Zezhou Cui, Liangchen Tan, Junkun Hong, Gehan Hu, Shuangyu Zhu, Yimou Wu, Shaohan Jiang, et al. RoboMemory: A brain-inspired multi-memory agentic framework for lifelong learning in physical embodied systems. *arXiv.org*, 2025. doi: 10.48550/arXiv.2508.01415.

Hao Li, Shuai Yang, Yilun Chen, Xinyi Chen, Xiaoda Yang, Yang Tian, Hanqing Wang, Tai Wang, Dahua Lin, Feng Zhao, et al. CronusVLA: Efficient and robust manipulation via multi-frame vision-language-action modeling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. doi: 10.1609/aaai.v40i22.38903.

Runhao Li, Wenkai Guo, Zhenyu Wu, Changyuan Wang, Haoyuan Deng, Zhenyu Weng, Yap-Peng Tan, and Ziwei Wang. MAP-VLA: Memory-augmented prompting for vision-language-action models in robotic manipulation. *arXiv.org*, 2025. doi: 10.48550/arXiv.2511.09516.

Li Lin, Wujun Xu, Wei Meng, Kaiwen Xia, K. Cheong, and Shuai Wang. FibVLA: An efficient temporal vision-language-action model with fibonacci sampling. *arXiv*, 2026.

Min Lin, Xiwen Liang, Bingqian Lin, Jingzhi Liu, Zijian Jiao, Kehan Li, Yuhan Ma, Yuecheng Liu, Shen Zhao, Yuzheng Zhuang, et al. Echovla: Robotic vision-language-action model with synergistic declarative memory for mobile manipulation. *arXiv.org*, 2025a. doi: 10.48550/arXiv.2511.18112.

Minghui Lin, Pengxiang Ding, Shu Wang, Zifeng Zhuang, Yang Liu, Xinyang Tong, Wenxuan Song, Shangke Lyu, Siteng Huang, and Donglin Wang. HiF-VLA: Hindsight, insight and foresight through motion representation for vision-language-action models. *arXiv.org*, 2025b. doi: 10.48550/arXiv.2512.09928.

Y. Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2022.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qian Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems 36*, 2023. doi: 10.52202/075280-1939.

Jiaming Liu, Mengzhen Liu, Zhenyu Wang, Lily Lee, Kaichen Zhou, Pengju An, Senqiao Yang, Renrui Zhang, Yandong Guo, and Shanghang Zhang. RoboMamba: Multimodal state space model for efficient robot reasoning and manipulation. *arXiv.org*, 2024a. doi: 10.48550/arXiv.2406.04339.

Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. RDT-1B: A diffusion foundation model for bimanual manipulation. *International Conference on Learning Representations*, 2024b. doi: 10.48550/arXiv.2410.07864.

Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. A survey on vision-language-action models for embodied AI. *IEEE Transactions on Neural Networks and Learning Systems*, 2024. doi: 10.1109/TNNLS.2025.3650584.

Marius Memmel, Jacob Berg, Bingqing Chen, Abhishek Gupta, and Jonathan Francis. STRAP: Robot sub-trajectory retrieval for augmented policy learning. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2412.15182.

Nvidia, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi-JimFan, Yu Fang, Dieter Fox, Fengyuan Hu, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv.org*, 2025. doi: 10.48550/arXiv.2503.14734.

Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. FAST: Efficient action tokenization for vision-language-action models. In *Robotics: Science and Systems XXI*, 2025. doi: 10.15607/rss.2025.xxi.012.

Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yani Ding, Zhigang Wang, Jiayuan Gu, Bin Zhao, Dong Wang, et al. SpatialVLA: Exploring spatial representations for visual-language-action model. In *Robotics: Science and Systems XXI*, 2025. doi: 10.15607/rss.2025.xxi.011.

Hongyu Qu, Jianzhe Gao, Xiaobin Hu, Shaohuan Yang, Xinlei Yu, Rui Yan, Wenguan Wang, Xiangbo Shu, and Shuicheng Yan. Dual latent memory in vision-language-action models for robotic manipulation. *arXiv*, 2026.

Ranjan Sapkota, Yang Cao, Konstantinos I. Roumeliotis, and Manoj Karkee. Vision-language-action models: Concepts, progress, applications and challenges. *arXiv.org*, 2025. doi: 10.48550/arXiv.2505.04769.

Yujiao Shen, Shulin Tian, Jingkang Yang, and Ziwei Liu. A simple baseline for streaming video understanding. *arXiv preprint arXiv:2604.02317*, 2026.

Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Feng Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. In *arXiv.org*, 2026a. doi: 10.48550/arXiv.2508.19236. arXiv:2508.19236.

Haochen Shi, Weiye Li, Bin Xie, Yulin Wang, Renping Zhou, Tiancai Wang, Xiangyu Zhang, Ping Luo, and Gao Huang. MemoryVLA++: Temporal modeling via memory and imagination in vision-language-action models. *arXiv.org*, 2026b. doi: 10.48550/arXiv.2606.09827.

A. Sridhar, Jennifer Pan, Satvik Sharma, and Chelsea Finn. MemER: Scaling up memory for robot control via experience retrieval. In *arXiv.org*, 2026. doi: 10.48550/arXiv.2510.20328.

Mingyang Sun, Xiujian Liang, Jiude Wei, Qichen He, Donglin Wang, Cewu Lu, and Jianhua Sun. Analytic concept-centric memory for agentic embodied manipulation. *arXiv.org*, 2026a. doi: 10.48550/arXiv.2606.29774.

Xiaoquan Sun, Ruijian Zhang, Chen Cao, Yihan Sun, Jiahui Chen, Zetian Xu, Bo Chen, Haijier Chen, Zhen Yang, Jiarun Zhu, et al. Himem-wam: Hierarchical memory-gated world action models for robotic manipulation. *arXiv preprint arXiv:2606.10363*, 2026b.

Shuhan Tan, Kairan Dou, Yue Zhao, and Philipp Krähenbühl. Interactive post-training for vision-language-action models. *arXiv.org*, 2025. doi: 10.48550/arXiv.2505.17016.

O. Team, Dibya Ghosh, H. Walke, Karl Pertsch, Kevin Black, Oier Mees, S. Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems XX*, 2024. doi: 10.15607/rss.2024.xx.090.

Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z Ren, Haohuan Wang, Jiaming Tang, Kyle Stachowicz, et al. Mem: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026.

Kai Wang, Zhaopeng Gu, Yixiang Chen, Yuan Xu, Qisen Ma, Peng Su, Zhaowen Li, Yan Huang, and Liang Wang. DIM-WAM: World-action modeling with diverse historical event memory. *arXiv.org*, 2026a. doi: 10.48550/arXiv.2606.27677.

Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Ke-Yang Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv.org*, 2024. doi: 10.48550/arXiv.2409.12191.

Ziye Wang, Modi Shi, Chaojun Ni, Jiazhi Yang, Mengdi Li, Zhizhong Su, Tianwei Lin, and Hongyang Li. NativeMEM: Native memory compression for long-horizon robotic manipulation. *arXiv*, 2026b.

Ronald J. Williams and Jing Peng. An efficient gradient-based algorithm for on-line training of recurrent network trajectories. *Neural Computation*, 1990. doi: 10.1162/neco.1990.2.4.490.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv*, 2025. doi: 10.48550/arXiv.2505.09388.

Ganlin Yang, Zhangzheng Tu, Yuqiang Yang, Sitong Mao, Junyi Dong, Tianxing Chen, Jiaqi Peng, Jing Xiong, Jiafei Cao, Jifeng Dai, et al. Eventvla: Event-driven visual evidence memory for long-horizon vision-language-action policies. *arXiv.org*, 2026a. doi: 10.48550/arXiv.2606.20092.

Sizhe Yang, Juncheng Mu, Tianming Wei, Chenhao Lu, Xiaofan Li, Linning Xu, Zhengrong Xue, Zhecheng Yuan, Dahua Lin, Jiangmiao Pang, et al. MemoryWAM: Efficient world action modeling with persistent memory. *arXiv.org*, 2026b. doi: 10.48550/arXiv.2606.20562.

Cheng Yin, Yankai Lin, Wang Xu, Sikyuen Tam, Xiangrui Zeng, Zhiyuan Liu, and Zhouping Yin. DeepthinkVLA: Enhancing reasoning capability of vision-language-action models. In *Third Conference on Language Modeling*, 2026.

Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv.org*, 2026. doi: 10.48550/arXiv.2603.16666.

Tony Zhao, Vikash Kumar, S. Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems XIX*, 2023. doi: 10.15607/rss.2023.xix.016.

Table 7: Per-benchmark instantiation of the SimpleMemVLA configuration tuple and the resulting training cost. Everything else about the method is identical across suites. Training time is wall-clock hours on 128 H100 GPUs.

| | RMBench | RoboMME | MIKASA-Robo | RoboMemArena | LIBERO |
|---|---|---|---|---|---|
| Embodiment | Aloha-AgileX, bimanual | Panda, single | Panda, single | Franka, single | Franka, single |
| Action dimension $d_a$ | 14 | 8 | 8 | 7 | 7 |
| History camera $\mathcal{C}_{\text{hist}}$ | head | front | overhead | front | agentview |
| Current cameras $\mathcal{C}_{\text{cur}}$ | both wrists | wrist | wrist | wrist | wrist |
| Window $T_w$ (s) | 60 | 60 | 3 | 126 | 30 |
| Video rate $f_v$ (fps) | 2 | 2 | 20 | 1 | 2 |
| Frames $K$ | 120 | 120 | 60 | 126 | 60 |
| Stride $s$ | 8 | 10 | 1 | 20 | 10 |
| Action horizon $H$ | 30 | 30 | 16 | 16 | 16 |
| Training time (h) | 23 | 20 | 16 | 28 | 14 |

Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayi Zou, Yilun Chen, Jia Zeng, et al. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *arXiv.org*, 2026. doi: 10.48550/arXiv.2510.10274.

Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daum'e, A. Kolobov, Furong Huang, and Jianwei Yang. TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2412.10345.

Shoujing Zhu, Zhenyang Liu, F. Wang, Jiafeng Wang, Bo Yue, Guiliang Liu, Si Wu, X. Xue, and Taiping Zeng. WeaveLA: Event driven cross-subtask latent memory weaving for repetitive robot manipulation. *arXiv.org*, 2026. doi: 10.48550/arXiv.2606.17463.

## A Implementation Details

Table 7 lists the per-benchmark instantiation of the configuration tuple from Section 3.4. All suites share the same backbone (Qwen3.5-4B, native video interface with native plaintext timestamps), the same conditioning rule and the same losses. Optimization uses AdamW with separate learning rates for the backbone and the action head ($10^{-5}$ and $5 \times 10^{-5}$), cosine decay and gradient clipping. The stride $s$ is the integer rounding of $f_c/f_v$.

**Sub-task labels.** The sub-task label $g_t^*$ of Section 3.3 is written offline by a cloud LLM: shown each demonstration, it states a one-sentence sub-task for every supervision anchor and the dataset stores the result per frame. The three mechanism variants of Section 4.3 are trained on these identical labels and the deployed policy never queries the annotator, so annotation is a training-data cost shared by every within-stack comparison and absent at inference.

**Baseline provenance and per-suite protocols.** RMBench third-party rows come from the papers cited in the text and EventVLA is quoted in its visual-anchors-only configuration. RoboMME rows follow the benchmark report, except the third-party WeaveLA on its own retrained $\pi_{0.5}$ stack. RoboMemArena rows follow the benchmark report, with the external FrameSamp+Modul entry quoted from the official leaderboard and evaluation run under the official protocol of 51 rollouts per task. MIKASA-Robo and LIBERO-Plus VLA baselines follow the MemoryVLA++ report (Shi et al., 2026b) and GMP numbers come from its Figure 11 with per-task training and tuned horizons. LIBERO numbers for Diffusion Policy, Octo, OpenVLA, $\pi_0$ and $\pi_0$-FAST follow the OpenVLA-OFT report (Kim et al., 2025) at 500 trials per suite and RIPT-VLA is quoted at its reported four-suite average. RMBench uses 100 evaluation seeds per task. Instructions are frozen to each benchmark's own task strings throughout, so no prompt is tuned per method.

Two engineering bounds delimit how far the native window stretches. The backbone's 262k position limit corresponds to about 48 min of history at the deployment sampling rate. The vision processor

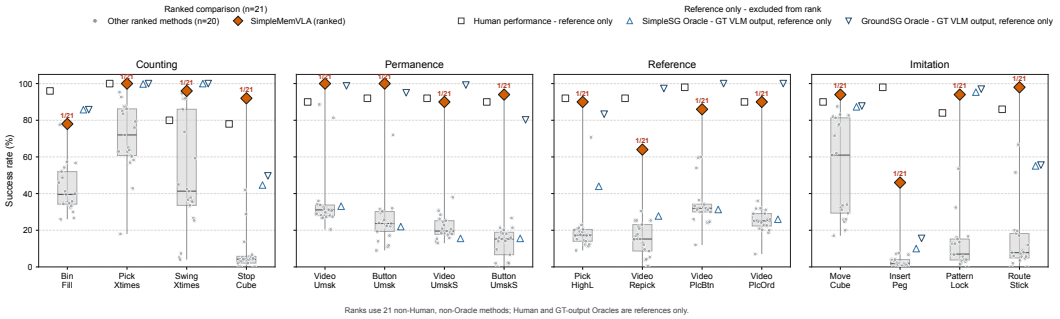



Figure 8: **First on all sixteen RoboMME tasks among the 21 deployable methods.** Gray dots denote the 20 other non-Human, non-Oracle methods evaluated by the benchmark. Boxes show the interquartile range of the 21-method ranked pool, with median and full range. Orange diamonds mark SimpleMemVLA and labels report its competition rank out of 21. Human performance and the two symbolic Oracle variants, which replace VLM outputs with ground-truth outputs, are shown only as references and are excluded from all ranks and distribution statistics.

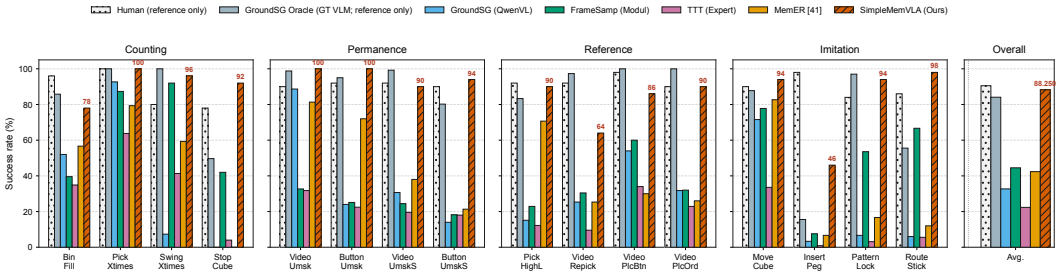



Figure 9: Task-wise RoboMME success rates (%, higher is better) for representative methods. Panels group the 16 tasks into Counting, Permanence, Reference and Imitation. Overall is recomputed from all 16 task scores. Human and GroundSG (Oracle, using ground-truth VLM outputs) are shown only as references and are excluded from the ranked comparison. For each remaining baseline family, the row with the highest overall AVG is shown. Values above orange hatched bars report SimpleMemVLA scores.

further enforces a total pixel budget that would shrink per-frame resolution once the window exceeds a few minutes, which we lift at runtime by scaling its longest-edge budget with the frame count so that every frame keeps the production resolution.

## B Full RoboMME Results

Table 8 reports the complete task-level RoboMME results underlying Table 2, including the per-task scores of our controlled variants, and Figure 4 plots the three rebuilds against the native stream. Figures 9 and 10 visualize the benchmark pool.




Figure 10: Complete task-wise RoboMME success rates (%, higher is better) for all 24 rows. Tasks follow the source-table order and methods retain the same order in every panel. Colors encode memory families. The orange hatched bar denotes SimpleMemVLA.

Table 8: Complete task-level RoboMME success rates (%). Human and GT-VLM Oracle rows (gray) are reference-only and excluded from ranking. **Bold** and underline mark ranks 1 and 2 among the remaining 21 methods. Task abbreviations follow Table 2's four dimensions in source order. The three SimpleMemVLA variant rows are the controlled within-stack rebuilds of Section 4.3 and are excluded from ranking.

| Memory | Method | Integr. / VLM | Counting | | | | Permanence | | | | Reference | | | | Imitation | | | | AVG |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | BinFill | PickX | SwingX | StopC | V-Umsk | B-Umsk | V-UmskS | B-UmskS | PickHL | V-Repk | V-PlcB | V-PlcO | MoveC | InsPeg | PatLock | RouteS | |
| Human | Human Performance | - | 96 | 100 | 80 | 78 | 90 | 92 | 92 | 90 | 92 | 92 | 98 | 90 | 90 | 98 | 84 | 86 | 90.5 |
| Symbolic (Oracle) | SimpleSG (Dai et al., 2026) | GT VLM | 85.78 | 99.78 | 100 | 44.67 | 33.11 | 22 | 15.56 | 15.56 | 44 | 27.78 | 31.33 | 26 | 87.33 | 10 | 95.33 | 55.11 | 49.58 |
| Symbolic (Oracle) | GroundSG (Dai et al., 2026) | GT VLM | 85.78 | 100 | 100 | 49.67 | 98.78 | 95 | 99.22 | 80.22 | 83.33 | 97.33 | 100 | 100 | 87.78 | 15.56 | 97 | 55.56 | 84.08 |
| Symbolic | SimpleSG | Gemini | 46 | 63 | 45 | 2 | 29 | 9 | 14 | 2 | 20 | 15 | 26 | 29 | 61 | 4 | 7 | 0 | 23.25 |
| Symbolic | SimpleSG | QwenVL | 77.56 | 95.33 | 5.11 | 0.44 | 34.22 | 19.33 | 15.33 | 9.56 | 17.11 | 25.33 | 33.33 | 25.11 | 82 | 3.78 | 12.67 | 7.78 | 29 |
| Symbolic | GroundSG | Gemini | 26 | 18 | 4 | 3 | 36 | 14 | 13 | 0 | 9 | 17 | 12 | 7 | 17 | 0 | 7 | 2 | 11.56 |
| Symbolic | GroundSG | QwenVL | 52 | 92.67 | 7.33 | 0 | 88.67 | 24 | 30.67 | 14 | 15.11 | 25.33 | 54 | 31.78 | 71.56 | 3.33 | 6.67 | 6 | 32.7 |
| Perceptual | TokenDrop (Dai et al., 2026) | Context | 48.67 | 85.11 | 94.67 | 3.11 | 33.78 | 31.56 | 26.22 | 16 | 20.67 | 17.78 | 31.11 | 25.33 | 81.33 | 4 | 12.67 | 20 | 34.5 |
| Perceptual | TokenDrop | Modul | 34.44 | 83.56 | 86 | 5.33 | 28.22 | 29.33 | 28.44 | 21.33 | 21.33 | 22 | 59.56 | 36 | 62 | 7.11 | 32.44 | 51.56 | 38.04 |
| Perceptual | TokenDrop | Expert | 54.22 | 87.56 | 91.78 | 4.22 | 26.67 | 30.44 | 18.44 | 18.89 | 19.33 | 20.89 | 36.44 | 24.67 | 87.56 | 2.22 | 16.22 | 18.22 | 34.86 |
| Perceptual | FrameSamp (Dai et al., 2026) | Context | 41.22 | 72 | 73.67 | 13.67 | 26.89 | 30.22 | 20.89 | 15.22 | 17.67 | 15.22 | 30 | 20.89 | 77.22 | 1.22 | 15.22 | 19.67 | 30.68 |
| Perceptual | FrameSamp | Modul | 39.56 | 87.33 | 92 | 42 | 32.67 | 25.11 | 24.44 | 18.22 | 22.89 | 30.44 | 60 | 32 | 77.78 | 7.56 | 53.56 | 66.67 | 44.51 |
| Perceptual | FrameSamp | Expert | 57.33 | 86.22 | 94.67 | 28.89 | 31.78 | 25.78 | 22.89 | 20.22 | 19.11 | 23.11 | 30 | 24.22 | 83.11 | 2 | 13.56 | 17.11 | 36.25 |
| Recurrent | TTT (Dai et al., 2026) | Context | 35.56 | 62.89 | 42.44 | 3.33 | 29.78 | 22.89 | 18.44 | 14.44 | 20.44 | 13.11 | 34.22 | 20.22 | 32.44 | 1.11 | 1.56 | 3.56 | 22.28 |
| Recurrent | TTT | Modul | 34.22 | 65.11 | 36.67 | 2.11 | 27.22 | 22.11 | 25.22 | 14.11 | 14.56 | 12.11 | 32.67 | 22.33 | 31.22 | 1.11 | 3.56 | 7 | 21.96 |
| Recurrent | TTT | Expert | 34.89 | 63.78 | 41.33 | 4 | 31.78 | 22.44 | 19.56 | 18 | 12.22 | 9.56 | 34 | 22.89 | 33.56 | 0.89 | 3.11 | 5.56 | 22.35 |
| Recurrent | RMT (Dai et al., 2026) | Context | 32.44 | 56.89 | 33.56 | 5.78 | 31.33 | 10.89 | 17.33 | 2 | 14 | 3.78 | 32 | 29.11 | 25.78 | 2 | 5.56 | 8.89 | 19.46 |
| Recurrent | RMT | Modul | 33.33 | 60.78 | 37.78 | 4.67 | 31.11 | 11.78 | 17.78 | 2.44 | 17.11 | 4.22 | 32 | 31.11 | 24.67 | 2.21 | 3.78 | 8 | 20.17 |
| Recurrent | RMT | Expert | 35.78 | 60.22 | 36 | 5.56 | 28 | 17.11 | 15.78 | 2 | 11.78 | 0.22 | 24.22 | 22.67 | 20 | 1.89 | 4.22 | 4.89 | 18.15 |
| Other | $\pi_{0.5}$ (Intelligence et al., 2025) | - | 30 | 42.89 | 35.56 | 6.67 | 20.44 | 22.22 | 18.67 | 6.67 | 11.33 | 0.44 | 31.11 | 25.78 | 26 | 1.56 | 2.89 | 4.67 | 17.93 |
| Other | $\pi_{0.5}$ w/ past actions (Dai et al., 2026) | - | 26.67 | 58.33 | 26.67 | 4.67 | 30.67 | 23.67 | 20.67 | 16 | 12.33 | 8.67 | 24 | 18.67 | 34 | 1 | 4 | 5.67 | 19.73 |
| Other | SAM2Act+ (Fang et al., 2025) | - | 40 | 76 | 25.33 | 0 | 27.33 | 32 | 18 | 26.67 | 17.33 | 5.33 | 24.67 | 20 | 29.33 | 0 | 0 | 0 | 21.37 |
| Other | MemER (Sridhar et al., 2026) | - | 56.67 | 79.33 | 59.33 | 0 | 81.33 | 72 | 38 | 21.33 | 70.67 | 25.33 | 30 | 26 | 82.67 | 6.67 | 16.67 | 12 | 42.38 |
| Ours variant | SimpleMemVLA (Retrieval) | - | 38 | 34 | 76 | 38 | 76 | 58 | 20 | 14 | 18 | 0 | 18 | 18 | 68 | 8 | 10 | 10 | 31.5 |
| Ours variant | SimpleMemVLA (Token Compression) | - | 26 | 50 | 92 | 16 | 18 | 12 | 14 | 10 | 16 | 6 | 16 | 12 | 56 | 0 | 10 | 8 | 22.625 |
| Ours variant | SimpleMemVLA (Recurrent State) | - | 26 | 36 | 56 | 6 | 26 | 24 | 30 | 16 | 12 | 4 | 34 | 20 | 16 | 0 | 10 | 14 | 20.625 |
| Ours | SimpleMemVLA | - | **78** | **100** | **96** | **92** | **100** | **100** | **90** | **94** | **90** | **64** | **86** | **90** | **94** | **46** | **94** | **98** | **88.25** |

Table 9: **LIBERO-Plus breakdowns.** Top: by source LIBERO suite. Bottom: by the benchmark's five difficulty levels, which stratify tasks by the accuracy of four reference models (Fei et al., 2025). The 121 tasks that carry no difficulty label in the benchmark metadata are omitted from the bottom panel.

| *By source suite* | | | | | |
|---|---|---|---|---|---|
| | Spatial | Object | Goal | Long | |
| Tasks | 2,402 | 2,518 | 2,591 | 2,519 | |
| Success (%) | 81.8 | 83.9 | 75.0 | 73.1 | |
| *By difficulty level* | | | | | |
| | Level 1 | Level 2 | Level 3 | Level 4 | Level 5 |
| Tasks | 1,644 | 2,202 | 2,094 | 1,886 | 2,083 |
| Success (%) | 90.5 | 88.1 | 83.4 | 74.3 | 56.3 |

Table 10: **The Sensor Noise column of Table 6, split by corruption type and severity tier.** Types are encoded in the benchmark's task naming. Success rates in %.

| Corruption type | Overall | Low | Mid | High |
|---|---|---|---|---|
| Motion blur | 87.8 | 100 | 95 | 76 |
| Glass blur | 85.7 | 93 | 89 | 79 |
| Gaussian blur | 69.2 | 95 | 78 | 47 |
| Zoom blur | 62.5 | 96 | 77 | 44 |
| Fog | 60.7 | 93 | 80 | 39 |

## C LIBERO-PLUS: PROTOCOL DETAILS AND BREAKDOWNS

**Protocol.** We follow the official LIBERO-Plus protocol (Fei et al., 2025): one trial per task over all 10,030 tasks (zero exclusions), a frozen initial state and fixed environment seed per task, ten no-op settling steps, per-suite step budgets of 220/280/300/520 and success judged by the simulator's built-in check. The seven dimensions differ in task counts (Camera 1,599, Robot 1,550, Language 1,537, Light 1,142, Background 1,076, Noise 1,601, Layout 1,525), so the Total in Table 6 is the success rate pooled over all tasks rather than a mean of the seven columns. One detail matters when reading the Language column: for the six non-language dimensions the official instruction string appends the perturbation tag to the task instruction (e.g., "... table 1"), which we feed verbatim. Only the Language dimension replaces the instruction with a rewritten one, so that column specifically measures robustness to paraphrase.

**By suite and difficulty.** Table 9 shows that the two more semantic suites degrade most under perturbation: relative to their unperturbed counterparts in Table 5, Goal loses 23.0 points (98.0 to 75.0) and Long 21.9 (95.0 to 73.1), versus 16.4 for Spatial and 14.9 for Object. Across the benchmark's difficulty stratification, success decays monotonically from 90.5% at Level 1 to 56.3% at Level 5. This graded degradation, rather than a bimodal solved/unsolved split, indicates that the 78.4% total is not carried by the easy strata alone.

**What the noise split reveals.** The 74.2% Sensor Noise column is not a uniform weakness. Split by corruption type (Table 10), the dividing line is not static versus dynamic corruption but whether the corruption preserves the geometric evidence in the visual stream. Motion blur and glass blur smear textures and edges while leaving the global scene layout intact and are largely absorbed (87.8% and 85.7%). Zoom blur superimposes rescaled copies that shift objects' apparent positions. Fog injects a per-frame random low-frequency luminance field that makes the history window temporally inconsistent. Gaussian blur at high severity erases object boundaries outright. Success collapses exactly on these three (62.5%, 60.7% and 69.2% overall, falling to 44%, 39% and 47% at the highest severity). All five types degrade monotonically with severity, indicating graded loss of evidence rather than a threshold artifact. The same criterion reads across the rest of Table 6: the dimensions SimpleMemVLA leads are those where scene geometry stays intact and the stream stays consistent

over the window (light, background and, through the relative-motion reference that a minute of history provides, robot initial states), while its weakest noise types are precisely the ones that attack positional evidence itself.

# D Additional Analysis

## D.1 The Gains Track Memory Load

If the gains came from a generically stronger backbone or better low-level control, they would be flat across memory loads. Instead the advantage grows along each suite's own memory axis (Tables 1, 2 and 3). On RMBench the margin over Mem-0 widens from +38.8 on single-memory tasks to +68.5 on multi-memory ones. On RoboMME it grows from +24.7 on Counting, whose periodic cues stay partly visible, to +44.5 on Reference, whose evidence is entirely off-frame. On MIKASA-Robo the lead over the best prior VLA stretches from 12 to 39 points as RememberColor moves from 3 to 9 candidates. Where memory is not the bottleneck the margin does not reverse but vanishes, since on LIBERO the same recipe is simply on par with the best reactive VLAs at 97.5%. This dose–response pattern ties the gains to memory itself.

The same lens delimits the claim. SimpleMemVLA's residual failures are consistently not retention failures: they concentrate in precision control (Insert Peg at 46%, where even the perfect-perception oracle reaches only 15.6%, Table 8), in fine-grained re-identification (Observe&Pickup 65%, RememberColor-5 58%, Video Repick 64%) and in perturbations that rewrite the instruction or attack positional evidence itself (Appendix C). Feeding the policy its own past removes the need for memory machinery. It does not substitute for perception or control.

## D.2 Why the Machinery Falls Short

The controlled comparison of Section 4.3 leaves a puzzle: the prior designs are far more elaborate than ours, yet they lose by wide margins. Three experiments locate the reason at write time, the moment such a mechanism must abstract the past before knowing what will matter.

The first is a controlled comparison that the RoboMME benchmark itself provides, since its fourteen memory variants share one $\pi_{0.5}$ backbone and differ only in the mechanism. The pool covers all three families of Figure 1, with external stores in both symbolic and retrieval form: symbolic scene-graph stores reach 11.6 to 32.7%, the retrieval-based MemER 42.4%, learned compression 30.7 to 44.5% and recurrent state 18.1 to 22.3%, against 88.3% for attending to the timestamped stream itself (Table 2). Our own rebuilds remove the last confound, since each shares everything with SimpleMemVLA except the memory interface, and no family survives the move onto our stack (20.6 to 31.5%, Section 4.3). That the rebuilds land at or below their families' pool rows reflects two different floors, not the mechanisms. The pool variants inherit the reactive competence of $\pi_{0.5}$, a generalist VLA pretrained on large robot corpora, and land within five points of it. Our stack learns control from this benchmark's demonstrations alone, so once the memory interface is broken there is no pretrained skill to fall back on. The mechanism's own contribution is the within-stack drop, from 88.3% to 31.5, 22.6 and 20.6 with everything else shared. No pretraining difference touches that comparison. More machinery does not help: the recurrent family's best variant stays at 22.3% and the lowest scores in the table belong to symbolic stores with a VLM in the write loop. The ordering holds beyond RoboMME as well: the strongest prior memory VLA reaches 44.4% on MIKASA-Robo where the raw stream reaches 74.0% and 73.1 against 78.4% zero-shot on LIBERO-Plus (Tables 3 and 6).

The second experiment removes write quality as the explanation. GroundSG handed ground-truth VLM outputs is a write-time abstraction executed perfectly, with no perception error and no capacity limit, yet it reaches 84.1%, below the 88.3% of the raw stream. The ceiling of this abstract-then-store pipeline sits below the input it abstracts, because what the write discards before the need is known stays missing at read time. Running the same pipeline with real perception loses another fifty points (32.7%) and the strongest deployable baselines collapse exactly where the needed evidence is a specific past percept: MemER falls to 38.0 and 53.2% on Reference and Permanence, Mem-0 from 52.8 to 28.5% as the number of facts grows and MemoryVLA++ from 97% on ShellGame to 16% on RC-9, where one cue must stay distinct from nine alternatives.

The third line of evidence comes from the deployment edits of Sections 4.4 and 4.6, which name what each abstraction discards. Shuffling frames zeroes both probe tasks and a retrieved set of keyframes is exactly an unordered set. The success cliff at the evidence age shows the read needs the stale frames that recency-biased compression evicts first. Zeroed timestamps halve counting and neither summaries nor recurrent states keep a time ruler. The graying interventions show the read consumes the percept itself, which no symbolic entry can reproduce. The recurrent rebuild compresses visual content, temporal order, timestamps and older evidence into a fixed sixteen-token state. It reaches 20.6% overall and is strongest on Counting at 31.0%, consistent with partial retention of running totals but not with reliable preservation of the perceptual and temporal evidence required across the suite. The keyframe bank discards only the order and the timestamps while keeping the percepts, so its 31.5% is both higher and selective in the way this list predicts, since the tasks it survives are the ones whose answer needs no time ruler. The compressor squeezes the percepts through a fixed budget, the very channel the graying interventions single out, and its 22.6% is selective in the complementary direction, keeping the tallies a summary can hold while the percept-dependent tasks the bank kept raw collapse. The one edit that never hurts is the control that keeps the evidence frames ordered and intact while removing others, so selection as such is not the problem. Discarding before the need is known is.

Nor does the dilemma leave a lossless way out. A write format that keeps the order, the timestamps, the stale frames and the percepts themselves has stopped abstracting, because up to the sampling rate it is the stream. A store that keeps everything and defers every choice to read time escapes the dilemma, but read-time selection over an intact past is what attention over the window already does, with no store to maintain. Attention over the raw stream wins by refusing the choice every tested family made: it defers selection to read time, when the need has arrived.

## E Bounded-Cost Streaming with Sliding-Window Attention

The exact streaming of Section 3.4 holds the *at-decision* compute at a single frame's cost, but the state behind it still grows with the stream: every softmax-attention layer appends one key–value block per temporal patch, so the cache and the background prefill grow linearly with the horizon and, beyond roughly ten minutes, the prefill begins to bound the sustainable decision rate (Figure 6b). This appendix describes a deployment variant whose per-decision compute *and* state are both constant in episode length. The policy is trained under a sliding-window-attention (SWA) regime whose deployed form is a single persistent token stream with block-wise cache eviction: nothing is ever recomputed and nothing grows. The variant rests on three observations.

**Window the softmax layers, and only them.** The backbone interleaves two kinds of sequence mixing: linear-attention layers, whose recurrent state is constant-size and summarizes the entire stream by construction, and softmax-attention layers, the only place where per-token state accumulates (24 and 8 of the 32 layers, respectively). SWA therefore touches nothing but the 8 softmax layers. During training and at deployment alike, each of their queries attends to at most the most recent $L$ tokens; the linear-attention layers are left untouched and keep integrating every token since the first frame of the episode. The window is sized in whole history units, where one unit is the token block of one temporal patch: its plaintext timestamp (6–8 tokens), the vision delimiters and 80 video tokens, 88–90 tokens in all. We set the window to $M = 60$ units, exactly the 60 s memory span the recipe already deploys on RMBench (Table 7), plus the decision suffix and the generation budget, giving $L = 5{,}888$ tokens with margin. Inside the window nothing changes: selection over the explicit past remains read-time attention, the thesis of Appendix D.2. Evidence older than the window survives only through the linear-attention state, the same finite-span caveat the windowed recipe always had, now with a compressed residue beyond it instead of nothing.

**Timestamps must be absolute.** A streamed token may never change after it is emitted, otherwise its cached key–value entries are invalid the moment the window slides. The window-relative timestamps of the deployed recipe violate exactly this: one decision later the same physical frame carries a label smaller by one execution interval, so every unit would need re-encoding at every step. The SWA variant therefore switches the timestamp convention from window-relative to episode-absolute, computed from each frame's index on the episode's sampling grid through one function shared by training and deployment. Under absolute time a unit's tokens are immutable, its cache

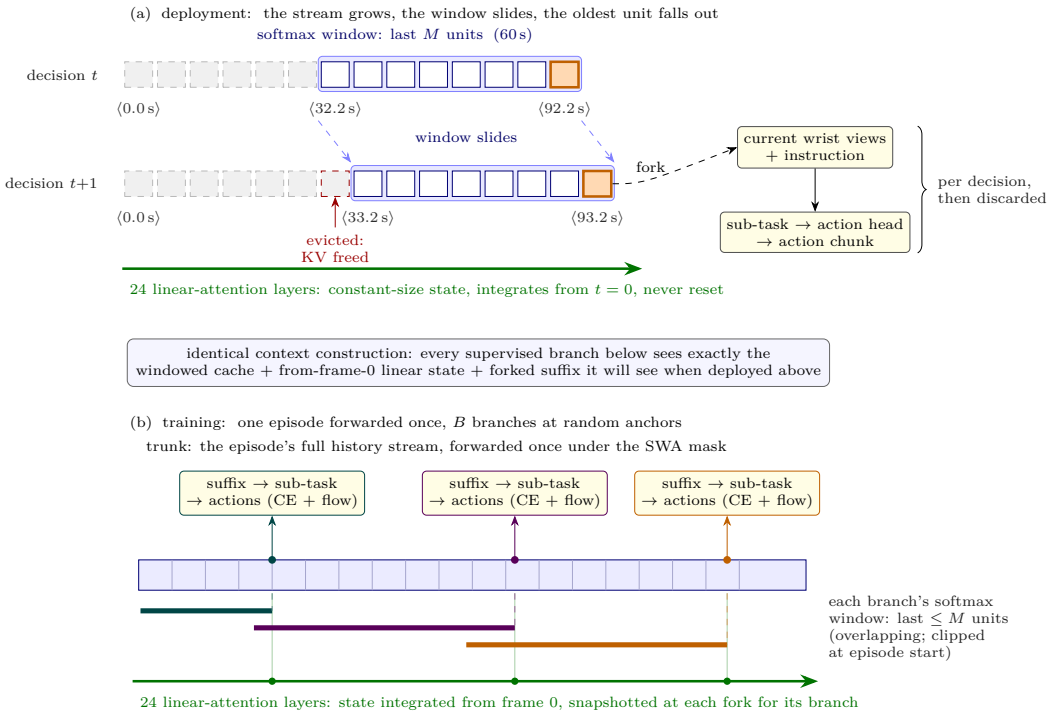



Figure 11: **The SWA variant: one context construction, trained and deployed.** **(a)** Two consecutive streaming decisions: each appends one unit (orange) to the persistent stream, the softmax window slides by one unit, and the unit leaving it (red) has its cache rows freed; the linear-attention state (green) integrates from $t = 0$ and is never reset. **(b)** Training forwards each episode once as a trunk and supervises $B$ branches at random anchors, each attending to its last $\leq M$ units and resuming the linear state snapshotted at its fork — exactly the deployed construction. Details in the accompanying text.

entries are written once and reused until evicted and positions continue monotonically without ever being renumbered.

**The conditioning interface needs no change.** The action head conditions only on the generated answer span and the state token (Equation 3), which sit at the tail of the sequence and hence always inside the window, so the train/deploy isomorphism of Section 3.3 carries over unchanged.

**Deployment: append one unit, evict one unit.** Figure 11a shows one decision. The persistent stream holds only the history video channel. When a decision fires, the newest temporal patch is encoded into one unit and appended (positions continue from the previous tail), softmax-cache rows older than the window are dropped in whole units and the linear-attention states are simply kept. The decision suffix, current wrist views, instruction and chat scaffold, is forked off the stream state, the sub-task is decoded, the action head conditions on its span and the fork is discarded. Eviction is exact rather than approximate: a dropped row is outside the window of every query that will ever be computed, so the streamed decision reproduces the full-prompt SWA decision token for token, which we verify at integer exactness for token and position ids and at kernel-noise level for activations. Per decision the variant therefore encodes one temporal patch, prefills one unit and decodes one short answer against a bounded cache: compute and memory are both independent of how long the robot has been running.

**Training with episode-level packing.** Training must expose the model to the same context construction, and the subtle point is the linear-attention layers. Ordinary per-anchor training renders each anchor a finite window and integrates the recurrent state from that window's first frame, while the streaming deployment integrates it from the episode's first frame, a train/deploy mismatch in

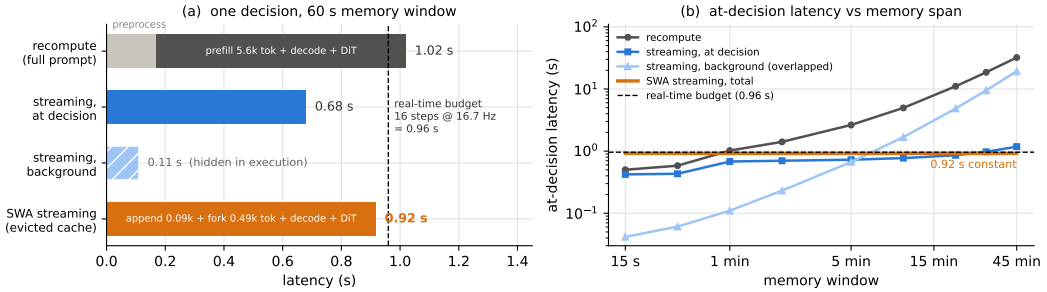



Figure 12: **Figure 6, extended with the SWA streaming variant** (measured on the longest RMBench episode; one H100, bf16, batch 1, calibrated against Figure 6's setup). **(a)** The whole SWA decision takes 0.92 s, inside the 0.96 s real-time budget, with no background work left to overlap. **(b)** Every prior mode's cost grows with the memory span; the SWA decision is constant, with the resident cache capped at the $L = 5{,}888$-token window.

Table 11: **Closed-loop success of the SWA streaming deployment**, on the RMBench protocol of Table 1 (100 held-out seeds per task).

| | Single-memory $M(1)$ | | | | | | Multi-memory $M(n)$ | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Method | Obs&PU | Rearr. | PutBack | SwapB | SwapT | *Avg* | Battery | Rank. | Cover | Press | Place-Mat$^{\dagger}$ | *Avg* | Overall |
| SimpleMemVLA (Table 1) | 65 | 100 | 100 | 100 | 93 | 91.6 | 90 | 100 | 98 | 100 | 100 | 97.0 | 94.0 |
| + SWA streaming (evicted cache) | 56 | 100 | 100 | 100 | 93 | 89.8 | 71 | 100 | 99 | 100 | 99 | 92.5 | 91.0 |

exactly the pathway SWA makes load-bearing. SWA training therefore packs samples at the episode level (Figure 11b). One sample is an (episode, phase) pair, where a phase is one residue of the temporal-patch grid, so that every anchor of the sample forks on a unit boundary of one shared trunk rendering. The trunk, the episode's full history stream, is forwarded once with the SWA mask on the softmax layers and the linear-attention state running from frame 0; $B = 8$ anchors drawn from the phase's chain fork off it, each resuming from the linear-attention states snapshotted at its fork and attending to the last $M$ units of trunk cache, and each pays the standard losses of Section 3.3. The packed pass is not an approximation: token and position ids match the per-anchor construction exactly and the supervised span's hidden states match within bf16 kernel noise, while the shared trunk cuts per-anchor compute several-fold. Random anchor draws with repeats cover every anchor near-uniformly across epochs. Relative to the recipe of Section 3, the SWA variant changes only the attention mask, the timestamp convention and the packing.

**Cost.** Figure 12 places the SWA variant on the axes of Figure 6. The exact streaming of the main text is what makes a *long-context* VLA deployable in real time: it moves the repeated prompt computation off the decision-critical path, holding the at-decision cost at the single-frame scale across minute-scale windows. What it does not bound is the state behind the stream — background prefill, cache memory and the consumed position range all grow with the accumulated history, the background prefill overruns the execution interval beyond roughly ten minutes, and just past 45 min the backbone's 262k position limit ends the episode outright. The SWA variant is what makes an *unbounded-context* VLA deployable: eviction caps the resident cache at the window, and although absolute positions keep growing, rotary attention only ever sees *relative* offsets bounded by the window, so the position limit never binds. Its measured decision — appending one ≈90-token unit, forking the ≈490-token suffix, decoding the sub-task and sampling the action head — is a constant 0.92 s at any memory span (median over 153 consecutive decisions of the longest, 146 s RMBench episode; the same bench reproduces Figure 6's fixed-window recompute at 0.94 s and its background prefill at 0.11 s, so the hardware is at par). Every quantity a deployment must budget for is therefore independent of how long the robot has been running, with memory beyond the window carried by the linear-attention state at zero marginal cost.

**Results.** Table 11 evaluates the SWA streaming deployment on the full RMBench protocol. Seven of the nine scored tasks stay within one point of the Table 1 deployment (five at 100%), and the overall score is 91.0 vs. 94.0. The remaining difference is concentrated in the two hardest memory tasks (Obs&PU and Battery) and reflects training under the SWA regime rather than the streamed inference itself, which reproduces the policy's full-prompt decisions exactly (verified at integer exactness for token and position ids and at kernel-noise level for activations, as argued above). The variant thus trades a small amount of accuracy on the hardest memory tasks for a deployment whose per-decision compute and memory are constant in episode length.